

技術解説書

OpenMX 4.0 GPU 版 実装解説書

NVIDIA GPU オフロード実装の全体像と
オリジナル OpenMX 4.0 からの全変更点
— cuBLAS・GEMMv8・cuSOLVER の使い方と注意点 —

対象リビジョン: OpenMX 4.0 GPU 版(2026-07-05 時点、コミット `8c7801fa`)

比較基準: オリジナル OpenMX 4.0 ソース一式

検証環境: NVHPC SDK 26.3 / CUDA 13.1 / compute capability 12.0(Blackwell 世代 GeForce)

発行日: 2026 年 7 月 5 日(第 2 版)

注記: 本書は第一原理計算コード [OpenMX](#)(GPL v3)を NVIDIA GPU 向けに移植した**非公式版**の技術解説書であり、OpenMX 開発チームによる公式文書ではありません。記載のファイル名・行番号は上記リビジョン時点のものであり、開発の進行に伴って変わる可能性があります。

目次

1. はじめに

2. 全体アーキテクチャ

設計方針 / レイヤ構成 / 有効化とデイスパッチ / GPU 化対象マップ / 複数ランク共有 GPU 対策

3. cuBLAS の使い方

使用ルーチン / ハンドルとストリーム / メモリ管理 / ソルバ別 GEMM エンジンの実像 / エラー処理 / 注意点

4. GEMMv8 の使い方

原理と出自 / 統合アーキテクチャ / 精度パラメータ / ワークスペース管理 / 複素対応 / ビルド / 注意点

5. cuSOLVER(gpuc solver 層)の使い方

層の構成と命名意図 / 使用 API / 一般化固有値問題 / devInfo 処理 / 並列度制御 / デバイス割当 / 注意点

6. OpenACC と生 CUDA の使い分け

データ常駐の 3 パターン / host_data 連携 / openmx_cuda_kernels.cu / NVHPC 固有の注意

7. 計算経路別の GPU 実装詳細

Band / Cluster / DC / DC-LNO / Krylov / 行列要素構築 / 全エネルギー / 力 / 派生ソルバ群

8. GPU 化以外の変更点

OpenMP バグ修正 / メモリ管理の堅牢化 / その他の修正

9. ビルドシステム

ツールチェーン / フラグ / リンク構成 / third_party 自動ビルド / ELPA / 派生 Makefile / 手順

10. 実行時制御リファレンス

入力キーワード / 環境変数一覧 / チューニング指針

11. 制約・注意点の総まとめ

付録 A. 変更ファイル一覧(37 ファイル)

付録 B. 新規追加ファイル一覧

付録 C. 開発履歴(全 51 コミット)

参考文献

1. はじめに

1.1 本書の目的と対象読者

本書は、密度汎関数理論(DFT)に基づく第一原理計算コード **OpenMX 4.0** を NVIDIA GPU 向けに移植した「OpenMX 4.0 GPU 版」について、(1) 現在の実装の全体像、(2) オリジナル OpenMX 4.0 からの**すべての変更点**、(3) 中核となる 3 つの GPU ライブラリ — **cuBLAS・GEMMu8・cuSOLVER** — の使い方と注意点、を一次情報(ソースコードと diff)に基づいて解説する公開ドキュメントです。想定読者は次のとおりです。

- OpenMX を GPU で高速に実行したいユーザー(第 2・10・11 章が中心)
- 本実装を理解・改修したい開発者(第 3～9 章)
- AMD GPU など他ベンダーへの移植を検討する開発者(第 5 章の gpusolver 層、第 11 章)

前提知識として、DFT・LCAO 法の初步、C 言語、MPI、CUDA プログラミングの基礎を仮定します。

1.2 OpenMX と本移植の位置づけ

OpenMX(Open source package for Material eXplorer)は、局在擬原子軌道(LCAO)基底とノルム保存擬ポテンシャルを用いた DFT コードで、クラスタ計算・バンド計算に加え、発散重畳(DC)法・DC-LNO 法・Krylov 部分空間法などの $O(N)$ 法、非平衡グリーン関数(NEGF)輸送計算などを備えます。ライセンスは GPL v3 です。

本 GPU 版は OpenMX 4.0 の公式ソースを出発点とし、**51 コミット**の開発で GPU オフロードを段階的に追加したものです。SCF 計算で支配的な「密行列の固有値問題」と「行列積(GEMM)」、および格子積分・縮約系の計算を GPU に移し、それ以外(疎行列処理、通信、入出力、CPU 固有経路)はオリジナルの構造を保っています。

1.3 変更規模の概要

項目	値	備考
変更されたソースファイル	37 ファイル(+ <code>makefile</code> → <code>Makefile</code> の改名・全面改修)	削除されたソースは 0
変更行数(生の diff)	+33,974 / -20,839 行	全面再インデント(<code>Force.c</code> 等)を含む。空白無視の実質変更は上位ファイルで各 1,000～3,700 行規模
新規ソースファイル	13 ファイル、計 2,555 行	gpusolver ラッパ、GEMMu8 ブリッジ、GPU 用エネルギーカーネルなど
新規ビルドファイル	3 ファイル	<code>Makefile</code> / <code>Makefile.bunshiken</code> / <code>makefile.cpu_intel.bak</code> (原本の同一保存)
third_party(git submodule)	2 件	GEMMu8(コミット固定)、FFTW 3.3.11(自前ビルド)

1.4 対象環境

- **コンパイラ**: NVIDIA HPC SDK(NVHPC)26.3 — `mpicc` (nvc)+ OpenACC、`mpif90` (nvfortran)、`nvcc`(CUDA 13.1)
- **GPU**: CUDA 対応 NVIDIA GPU。GEMMu8 は INT8 Tensor Core を使用。開発・検証は compute capability 12.0(Blackwell 世代 GeForce)で実施
- **並列モデル**: MPI プロセス並列。ノード内 local rank によるラウンドロビンで GPU を割当て、**複数ランクが 1 GPU を共有**することを前提に設計
- **OpenMP**: ビルドは `-fopenmp` 付きだが、GPU 経路の一部(`Set_OLP_Kin` の動径積分オフロード)は 1 スレッド実行を要求する。GPU 利用時は MPI のみ(スレッド数 1)の運用が想定されている

1.5 表記法

- ソース位置は `ファイル名:行番号` で示す(すべて `source/` ディレクトリ基準、2026-07-04 時点)。
- **GPU** 常時 GPU 実行、**条件付き** 環境変数などによるオプトイン、**CPU** CPU 実行(意図的に GPU 化しない)、**休眠** 実装済みだが現行経路では呼ばれないコード。
- 「オリジナル」は OpenMX 4.0 公式配布のソースを指す。

本書の読み方: ライブラリの使い方だけを知りたい場合は第 3～5 章、運用パラメータは第 10 章、既知の落とし穴は第 11 章にまとまっています。第 7 章は計算経路(ソルバ)別の縦割り解説で、第 3～6 章(ライブラリ・技法別の横割り)と相互参照になっています。

2. 全体アーキテクチャ

2.1 設計方針

1. **実行時切替・CPU 経路の完全温存** — GPU 化は `#ifdef` による一括置換ではなく、既存コードへの**分岐追加**で実現している。入力ファイルの 1 行 `scf.eigen.lib gpusolver` だけで有効化され、指定しなければオリジナルと同じ ELPA/ScaLAPACK/LAPACK 経路が走る。ゲートは実行時フラグ `scf_eigen_lib_flag == GPUSOLVER` (enum 値 3、`openmx_common.h:2807-2812`) の一点に集約されている。
2. **大規模な密行列問題のみ GPU 化** — 行列次元がしきい値 `GPU_CPU_SWITCH_NUM = 1000` (`openmx_common.h:4079`、DC-LNO のみ独自に 800) 未満の場合は転送コストが勝るため CPU にフォールバックする。
3. **MPI ランク単位の GPU 利用と「共有」前提の設計** — ノード内 local rank % GPU 数のラウンドロビン割当。1 GPU を複数ランクが共有する状況を正面から想定し、GPU ターン直列化・適応並列度制限・メモリ見積りによる事前判定・失敗時リトライという多層の制御を持つ (§ 2.5)。
4. **GEMM は GEMMv8 を第一選択、cuBLAS FP64 へ自動フォールバック** — 大規模 GEMM は INT8 Tensor Core による FP64 エミュレーション (Ozaki Scheme II) で実行し、ワークスペースが確保できない場合のみ native cuBLAS に切り替える (第 4 章)。
5. **対角化は cuSOLVER の 64-bit generic API** — `cusolverDnXsyevdx` (部分固有値) を中心に使い、一般化固有値問題 $Hc = \epsilon Sc$ は S の固有分解によるレイディン (対称) 直交化を GPU 上で自前実装する (第 5 章)。
6. **ベンダー中立の命名 (gpusolver)** — 将来の AMD GPU 対応を見据え、ラッパ層の識別子・入力キーワードを `cusolver` から `gpusolver` へ一括改名済み (コミット 2302f965, d8219384)。cuSOLVER API の呼び出し自体は NVIDIA ビルドの実装詳細として残る。なお開発初期に MAGMA も評価されたが、現在はソースツリーから完全に削除されている (コミット 788acb4c 「Delete unnecessary files, such as the MAGMA library」)。現行ソースに MAGMA への参照は 0 件)。
7. **数値再現性への配慮** — 密度行列構築の縮約を `#pragma acc loop seq` で逐次順序に保つ、球ベッセル関数テーブルを bit パターン一致キーでキャッシュする、GEMMv8 自体が bitwise 再現性を持つ、など加算順序への配慮が随所にある (一方で atomic 加算による順序非決定箇所も存在する。§ 11 参照)。

2.2 レイヤ構成

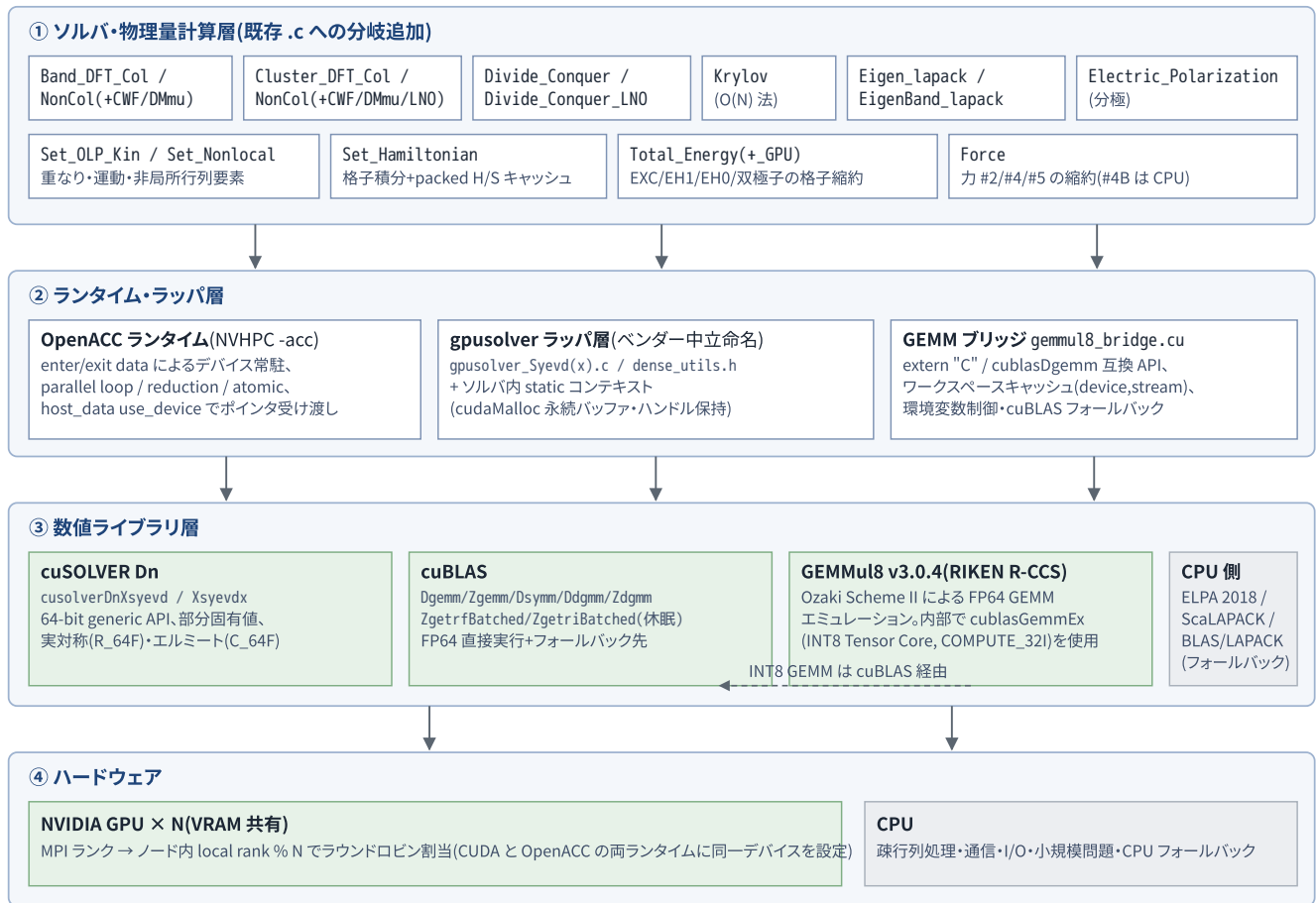


図 1: OpenMX 4.0 GPU 版のレイヤ構成。緑は GPU 実行、灰は CPU 実行。

GPU 実行の形態は 3 種類が併用されている。

1. **OpenACC ディレクティブ** — 格子積分・縮約・行列組立などのループ並列。データ常駐は `enter data / exit data` (第 6 章)。
2. **ソルバ内 static コンテキスト+生 CUDA API** — `cudaMalloc` による永続デバイスバッファと cuBLAS/cuSOLVER ハンドルをファイルスコープの構造体に保持し、ライブラリを直接呼ぶ(`BandColGpuSolverCtx` など。第 3・5 章)。
3. **extern "C" ブリッジ経由の GEMM v3.0.4** — C コードから C++ テンプレートライブラリを呼ぶための単一翻訳単位方式(第 4 章)。

2.3 有効化の流れと SCF 内の GPU 化ポイント

GPU 経路が有効になるまでの流れは次のとおり。

1. **入力解釈**(`Input_std.c:777-791`): `scf.eigen.lib gpusolver` で `scf_eigen_lib_flag = GPUSOLVER`。旧名 `cusolver` は非推奨エイリアスとして警告付きで受理される。ユーザー指定値は `scf_eigen_lib_flag_input` に保存。
2. **デバイス初期化**(`DFT.c:52-99`、SCF 入口 `DFT.c:334` の `DFT_GPU_DeviceInit()`): `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内 local rank を求め、`cudaSetDevice` と `acc_set_device_num` を対で設定する。CUDA/OpenACC デバイスが見つからなければ警告を出して `scf_eigen_lib_flag = ELPA2` に自動降格する。
3. **各ソルバ入口での条件判定**: `GPUSOLVER` フラグ && 行列次元 ≥ しきい値 &&(経路によっては)行列分散形状の条件を満たすときだけ GPU 密行列経路に入り、満たさなければ従来経路(ELPA2 等)がそのまま走る。

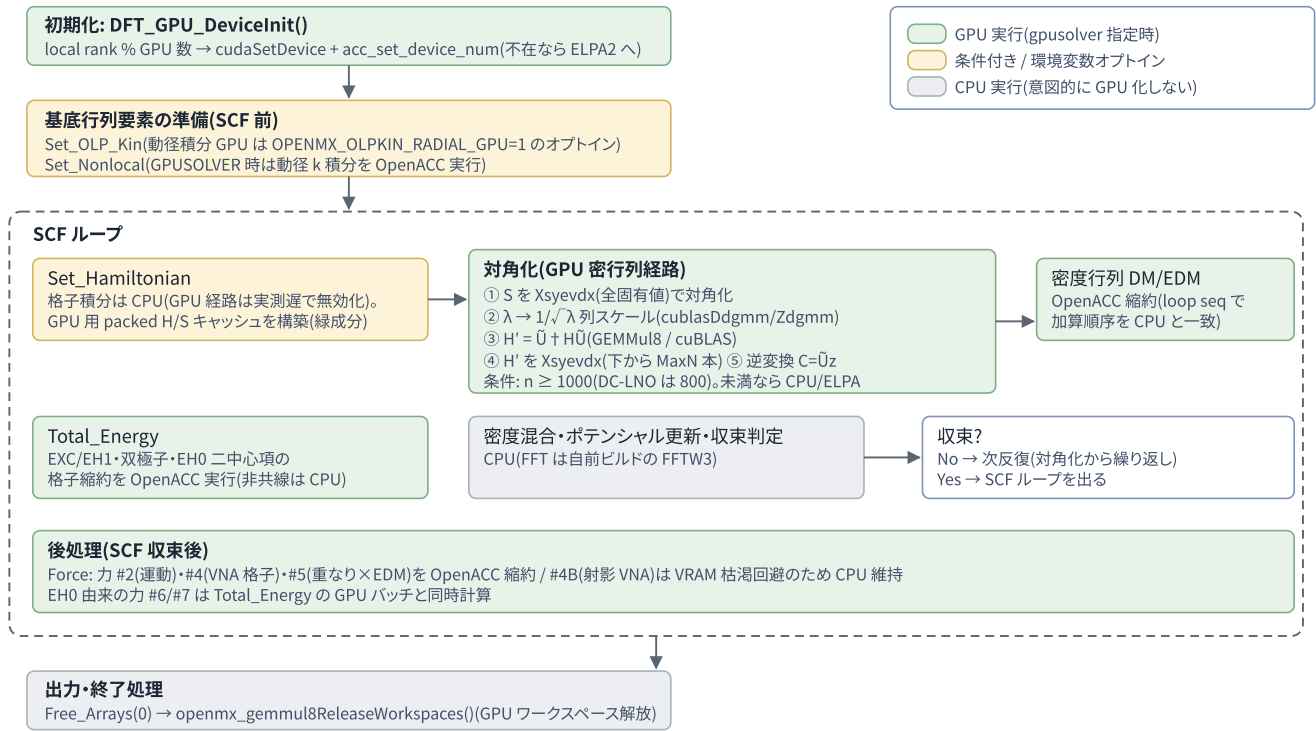


図 2: SCF 計算 1 サイクルの流れと GPU 化ポイント(コリニア・バンド/クラスタ計算の場合)。

2.4 GPU 化対象マップ

計算経路(scf.EigenvalueSolver)別・補助計算別の GPU 化状況を表 2-1 に示す。詳細は第 7 章。

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
クラスタ(Col)	Cluster_DFT_Col.c	root ランクに密行列を集約し、S 対角化 → レイディン直交化 → H' 対角化 → 逆変換を GPU 実行(root-dense 経路)。GEMM は GEMMu8 Dgemm	GPUSOLVER かつ $n \geq 1000$
クラスタ(NonCol)	Cluster_DFT_NonCol.c	root-dense 対角化(Dsyevx/Zheevx) + $U + HU$ を GEMMu8 で 12 回 + DM/EDM 縮約。固有ベクトルをキャッシュし DFT.c から scatter	GPUSOLVER かつ $2n \geq 1000$
バンド(Col)	Band_DFT_Col.c	k 点ごとの密行列対角化一式をデバイス常駐で実行。GEMM は GEMMu8 Zgemm。S 変換行列をキャッシュ。all_knum==1 / !=1 の両経路対応。GPU ターン直列化	GPUSOLVER かつ $n \geq 1000$ (+分散形状条件)
バンド(NonCol)	Band_DFT_NonCol.c	OpenACC 常駐型 root-dense 経路。GEMM は GEMMu8 Zgemm。VRAM 見積りによる適応並列度制限	GPUSOLVER かつ $2n \geq 1000$ (+分散形状条件)
DC 法($O(N)$)	Divide_Conquer.c	原子ごとの部分クラスタ行列を Xsyevdx で対角化。GEMM は GEMMu8→cuBLAS→CPU の多段フォールバック。VRAM 事前判定	GPUSOLVER かつ局所次元 ≥ 1000 (OPENMX_DC_GPU_THRESHOLD で変更可)
DC-LNO 法	Divide_Conquer_LNO.c	各ランクが自分の局所固有値問題を直接 GPU に投入(direct per-rank dispatch)。GPU プロキシ機構も温存(既定無効)	GPUSOLVER かつ局所次元 ≥ 800
Krylov 法($O(N)$)	Krylov.c	スレッド別ワークスペースプールによる生 CUDA 実装。部分空間対角化(Xsyevd/Xsyevdx)+ 大きい dgemm のみ GEMMu8	GPUSOLVER かつ $n \geq 1000$ (GEMM は $m \cdot n \cdot k \geq 5 \times 10^7$)

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
汎用固有値ルーチン	<code>Eigen_lapack.c</code> <code>EigenBand_lapack.c</code>	$n \geq 1000$ で <code>gpusolver_Syevdx(_Complex)</code> へ分岐。これらと呼ぶ全 20 以上のファイルが自動的に GPU 化の恩恵を受ける	GPUSOLVER かつ $n \geq 1000$
分極・DMmu・CWF・LNO 系	11 ファイル	ELPA2 分岐の手前に定型の GPUSOLVER 分岐(<code>dense_utils</code> 経由の <code>Dsyevx/Zheevx</code>)を挿入	GPUSOLVER かつ $n \geq 1000$ かつ <code>na_rows==n</code> 等
行列要素構築	<code>Set_Hamiltonian.c</code> <code>Set_Nonlocal.c</code> <code>Set_OLP_Kin.c</code>	非局所射影の動径積分は OpenACC 実行。 <code>Set_Hamiltonian</code> の格子積分 GPU 化は実測で遅く無効化(<code>packed H/S</code> キャッシュ構築が GPU ソルバを支える)。 <code>Set_OLP_Kin</code> はオプティン	GPUSOLVER(+ファイル別条件、§ 7.7)
全エネルギー	<code>Total_Energy.c</code> <code>Total_Energy_GPU.c</code>	EXC/EH1・双極子・CWF-Dc・EH0 二中心項の格子縮約を OpenACC 実行	GPUSOLVER かつ <code>SpinP_switch≠3</code>
力	<code>Force.c</code>	力 #2/#4/#5 の縮約を OpenACC 実行。#4B は CPU 維持	GPUSOLVER(+項別条件)
NEGF・複素逆行列	<code>Lapack_LU_inverse.c</code>	休眠 <code>cublasZgetrf/ZgetriBatched</code> による GPU 版 LU 逆行列を実装済みだが呼び出しはコメントアウト	—(将来の布石)

表 2-1: GPU 化対象マップ

2.5 複数ランク共有 GPU への対策(概観)

本実装の特徴は、「16 GB 級の GPU 1 台を 8 MPI ランクで共有する」ような構成を明示的に想定した多層防御にある。

機構	内容	実装箇所
アロケーション失敗リトライ	<code>wait_cudafunc</code> マクロが CUDA/cuBLAS/cuSOLVER 呼び出しを成功までランダム待ち($\leq 10 \mu\text{s}$)付きで再試行。他ランクとの一時的な VRAM 確保競争を吸収	<code>openmx_common.h:4092-4103</code>
GPU ターン直列化	バンド計算で「スピン $\times$ k 点」の通し番号(ターン)を導入し、既定 4 ターンごとに <code>MPI_Barrier</code> で区切って直列実行(8 ランク/GPU では並行投入より約 27% 高速というコメントあり)	<code>Band_DFT_Col.c:218-355</code>
適応並列度制限	非共線バンドでは必要 VRAM を明示的に見積り、 <code>cudaMemGetInfo</code> と MPI 合意で同時実行数を $4 \rightarrow 2 \rightarrow 1$ に自動縮小(環境変数による明示指定値にも適用)	<code>Band_DFT_NonCol.c:729-1038</code>
ワークスペースのターン毎解放	約 1 GiB の GEMMu8 ワークスペースをターン終了ごとに返却(既定有効)	<code>Band_DFT_Col.c:283-295, 2703-2705</code>
事前メモリ判定+CPU フォールバック	DC 法は確保前に空き VRAM を検査し、不足なら GPU 経路を(sticky に)無効化して CPU で継続。 <code>Set_Hamiltonian</code> も共有ランク合算の <code>Allreduce</code> 判定を持つ	<code>Divide_Conquer.c:833-880</code> <code>Set_Hamiltonian.c:164-268</code>
密ソルバ実行ランクの選抜	クラスタ/バンドでは world 代表ランクだけが密行列 GPU 経路のオーナーになる	<code>DFT.c:159-204</code>

休眠コードの扱い: 本書では、定義は存在するが現行経路では呼ばれないコード (`gpusolver_Syevd`、`LU_Zinverse_GPU`、`openmx_cuda_kernels.cu`、一部の GEMMu8 ヘルパなど)を **休眠** と明示して区別する。一覧は § 11.7 を参照。

3. cuBLAS の使い方

3.1 使用ルーチンと役割

cuBLAS は本実装で 3 つの役割を持つ: (1) 対角スケーリング(`Ddgmm` / `Zdgmm`)など GEMM 以外の密行列演算の直接実行、(2) GEMM_{ul}8 が使えないときの FP64 フォールバック先、(3) GEMM_{ul}8 内部の INT8 Tensor Core GEMM(`cublasGemmEx`)の実行基盤。かつて一部ソルバに残っていた `cublasDgemm/Zgemm/Dsymm` の直接呼び出しは、コミット 66ce1e62 (Use GEMM_{ul}8 for remaining GPU GEMM paths)ですべて **GEMM_{ul}8 経由に統一**された。使用ルーチンの全体像を表 3-1 に示す。

cuBLAS ルーチン	用途	主な使用箇所
<code>cublasDgemm</code> / <code>cublasZgemm</code>	GEMM _{ul} 8 ブリッジ内の FP64 フォールバック先、および DC 法の多段フォールバック 2 段階(直接呼び出しはこの 2 箇所のみ)	<code>gemmul8_bridge.cu:299,307,373,381</code> 、 <code>Divide_Conquer.c:607</code>
<code>cublasDdgmm</code> / <code>cublasZdgmm</code>	レイディン直交化の $1/\sqrt{\lambda}$ 列スケーリング(対角行列との積)	<code>Cluster_DFT_Col.c:261</code> 、 <code>Band_DFT_Col.c:1377</code> 、 <code>Divide_Conquer_LNO.c:520</code>
<code>cublasZgetrfBatched</code> / <code>cublasZgetriBatched</code>	休眠 複素 LU 分解・逆行列(batch=1)。NEGF 表面グリーン関数向けの布石(§5.8)	<code>Lapack_LU_inverse.c:384,400</code>
<code>cublasGemmEx</code> (INT8, COMPUTE_32I)	GEMM _{ul} 8 内部の法別 INT8 行列積(OpenMX からは直接呼ばない)	<code>third_party/GEMM_{ul}8/src/oz2/common/matmult.hpp:16-25</code>

表 3-1: 使用 cuBLAS ルーチン一覧

3.2 ハンドルと CUDA ストリームのライフサイクル

現行実装の主経路では、ソルバごとのファイルスコープ **static コンテキスト**が cuBLAS/cuSOLVER ハンドルを 遅延初期化で 1 個ずつ保持する(MPI プロセスあたりソルバ種別ごとに 1 個)。デバイスが変わった場合のみ破棄・再生成される。

Cluster_DFT_Col.c:118-130 — コンテキストの遅延初期化(典型形)

```
wait_cudafunc(cudaGetDevice(&current_device));

if (ctx->initialized && ctx->device_id == current_device) return;
if (ctx->initialized) ClusterCol_GpuSolver_Destroy();

wait_cudafunc(cudaStreamCreateWithFlags(&ctx->stream, cudaStreamNonBlocking));
wait_cudafunc(cublasCreate(&ctx->cublas));
wait_cudafunc(cusolverDnCreate(&ctx->gpusolver));
wait_cudafunc(cublasSetStream(ctx->cublas, ctx->stream));
wait_cudafunc(cusolverDnSetStream(ctx->gpusolver, ctx->stream));

ctx->initialized = 1;
ctx->device_id = current_device;
```

ストリームの扱いはソルバ間で**非対称**である点に注意が必要である。

- **Cluster_DFT_Col / Band_DFT_NonCol(DM 用) / Krylov / DC / DC-LNO:** 専用の non-blocking ストリームを生成し、`cublasSetStream` / `cusolverDnSetStream` で束ね、転送は `cudaMemcpyAsync` + 要所の `cudaStreamSynchronize` で順序を保証する。
- **Band_DFT_Col:** コンテキスト構造体(`Band_DFT_Col.c:1105-1132`)に stream メンバ自体がなく、`cublasSetStream` も呼ばれない。cuBLAS はデフォルトストリームで動き、転送も同期 `cudaMemcpy` 。

- **OpenACC との順序制御**: OpenACC の既定キューとライブラリ用ストリームは束ねていない (`acc_get_cuda_stream / async` 節は全コードで不使用)。OpenACC 管理データをライブラリに渡す直前に `#pragma acc wait`、直後に `cudaStreamSynchronize` を置いて明示同期する (例: `Band_DFT_NonCol.c:433-455`)。

注意: ソルバ間でストリーム前提が異なるため、あるソルバのヘルパ関数を別ソルバに移植する際に同期規約を混ぜると競合の恐れがある。また、各コンテキストのハンドル・永続バッファは**プログラム正常終了時に明示破棄されない**(解放されるのはデバイス変更時と Band 系のターン終了時のみ。終了時は CUDA コンテキスト破棄任せ)。 `Free_Arrays(0)` が解放するのは GEMMu8 ワークスペースだけである。

3.3 デバイスメモリ管理

- **grow-only の永続バッファ**: `if (n <= ctx->matrix_dim) return;` の形で、必要サイズが既存以下なら再利用し、不足時のみ free→malloc する (`Cluster_DFT_Col.c:146-164`、 `Band_DFT_Col.c:1221-1248`)。cuSOLVER ワークスペースも同様 (`Cluster_DFT_Col.c:192-208`)。
- **pinned メモリ**: ほぼ不使用。唯一 DC 法がホスト側に `cudaMallocHost` を使う (`Divide_Conquer.c:786`)。他はページャブル malloc + (Async) Malloc。
- **共有 GPU への配慮**: Band 系の SCF 反復間バッファ常駐は既定 OFF (`OPENMX_BAND_GPU_PERSISTENT`)。コード内コメントは「16 GB を 8 ランクで共有すると GEMMu8 が FP64 フォールバックに追い込まれ、反復あたり 0.2 秒の節約より高つく」と明記する (`Band_DFT_Col.c:233-237`)。

3.4 ソルバ別 GEMM エンジンの実像

コミット `66ce1e62` 以降、GPU 上で実行される GEMM は全ソルバとも GEMMu8 ブリッジ経由に統一された (cuBLAS FP64 が使われるのはブリッジ内フォールバックと DC 法の多段フォールバックのみ)。ただし定義だけ存在して呼ばれないヘルパが複数残るため、grep の見かけに惑わされないよう、実際に呼ばれる経路(ライブ経路)を表 3-2 に整理する。

ソルバ	ライブ経路の GEMM エンジン	備考(休眠コード)
Band_DFT_Col	GEMMu8 (<code>openmx_gemm8Zgemm</code> × 3、三重積変換と逆変換。 <code>Band_DFT_Col.c:1452,1456,1473</code>)	ヘルパ <code>BandCol_GpuSolver_Zgemm_OpenACC (:1398)</code> は未使用
Band_DFT_NonCol	GEMMu8 (<code>BandNonCol_GEMMu8Zgemm_OpenACC :433</code> 経由 × 6。 <code>66ce1e62</code> で cuBLAS 直接から切替)	永続ワークスペースの cublas ハンドル+ストリームを使用。 <code>lda/ldb</code> は転置対応で計算
Cluster_DFT_Col	GEMMu8 (<code>ClusterCol_GEMMu8Dgemm_Device :285</code> 。H' 構成 × 2 + 逆変換 × 1。 <code>66ce1e62</code> で <code>cublasDsymm+Dgemm</code> から切替 — SYMM の対称性利用は廃止)	OpenACC 版ヘルパ(<code>:333</code>)は定義のみ。旧経路 <code>ClusterCol_GpuSolverDensePath</code> (CPU <code>dgemm</code> 使用)は到達不能
Cluster_DFT_NonCol	GEMMu8 (D/Z 合計 12 回、U + HU 変換)	—
Divide_Conquer(DC)	GEMMu8 → cuBLAS → CPU の多段 (<code>DC_GpuSolver_TryGpuDgemm</code> 、 <code>Divide_Conquer.c:575-609</code>)	GEMMu8 失敗時は以後そのランクで GEMMu8 を恒久無効化
Divide_Conquer_LNO	GEMMu8 (<code>openmx_gemm8Dgemm</code> × 3)	—
Krylov	GEMMu8 (<code>openmx_gemm8Dgemm</code> 、 <code>Krylov.c:346</code> 。 $m \cdot n \cdot k \geq 5 \times 10^7$ のときのみ GPU)	<code>66ce1e62</code> で <code>cublasDgemm</code> から切替
DMmu/CWF/LNO 系 11 ファイル	GPU は対角化のみ(GEMM は従来どおり)	各ファイルの GEMMu8 ヘルパは定義のみ(準備扱い)

表 3-2: ソルバ別 GEMM エンジン(ライブ経路と休眠コードの区別)

3.5 撤去された汎用ラッパ my_cublasDgemm / my_cublasZgemm

初期開発で使われた自己完結型 GEMM ラッパ my_cublasDgemm.c / my_cublasZgemm.c (呼び出しごとに cublasCreate → cudaMalloc / cudaMemcpy → GEMM → 破棄)は、長らく呼び出し箇所 0 件の休眠コードだったのち、コミット a7f8cba6 (Remove obsolete cuBLAS wrapper files)でソース・宣言・リンク対象とも完全に削除された。古いリビジョンを読む際にのみ登場する名前である。

関連して注意すべき簡約が現行ヘルパにも残る: ソルバ内 GEMM ヘルパは **alpha=1.0 / beta=0.0 固定**であり、lda/ldb の扱いは 2 系統ある。BandNonCol_GEMMu18Zgemm_OpenACC / ClusterCol_GEMMu18Dgemm_OpenACC は 66ce1e62 で転置に応じた lda/ldb 計算に修正済みだが、Cluster_DFT_NonCol.c:660,677 のライブヘルパ (正方行列でのみ使用)や DMmu/CWF 系の休眠ヘルパには **lda=m・ldb=k のハードコード**が残っており、転置指定時に正しいのは正方行列(m=n=k)の場合だけである。非正方・転置で再利用する場合は ld の扱いを必ず確認すること。

3.6 エラー処理 — wait_cudafunc マクロ

CUDA/cuBLAS/cuSOLVER 呼び出しのほぼすべて(Band_DFT_Col.c だけで 47 箇所)が次のマクロで包まれる。

openmx_common.h:4079-4103(抜粋)

```
#define GPU_CPU_SWITCH_NUM 1000
#define WAITTIME (10.0 * 1.0E-6)

#define wait_cudafunc(call) \
    while (true) { \
        if (cudaSuccess == call) { \
            break; \
        } \
        double wait_time = drand48() * WAITTIME; \
        double start_time = MPI_Wtime(); \
        double current_time = start_time; \
        while ((current_time - start_time) < wait_time) { \
            current_time = MPI_Wtime(); \
        } \
    }
```

失敗時は 0~10 μs のランダム待ち(ビジーウェイト)を挟んで**成功するまで同じ呼び出しを無限に再試行**する。成功コード 0 は cudaError_t / cublasStatus_t / cusolverStatus_t に共通なので 3 種の API を同一マクロで包める。これは複数ランクが同一 GPU を取り合う際の一時的な cudaMalloc 失敗を吸収するための設計である。abort 型の cudacall マクロ (openmx_common.h:4082-4090)も定義されているが、主要経路では使われていない。

計算失敗(固有値の収束失敗など)への対応はソルバ層で行われ、経路により MPI_Abort / exit か CPU フォールバックかが異なる(詳細は §5.4 の表)。

3.7 cuBLAS 使用上の注意点

- wait_cudafunc は無限リトライ** — 引数の呼び出し式は每周再評価される。恒久的エラー (不正な leading dimension、デバイス喪失など)を渡すとハング+副作用の反復になる。GPU 実行が「止まったように見える」ときは まずここを疑う。
- 簡約ヘルパの ld/alpha/beta 固定** — §3.5 のとおり。ライブヘルパの lda/ldb は転置対応に修正済みだが、一部ヘルパに固定が残り、転置 + 非正方の組み合わせで静かに誤結果になる。
- ストリーム前提の非対称性** — Band_DFT_Col はデフォルトストリーム+同期 memcpy、他は専用ストリーム+明示同期。コードを移植・共通化する際は同期規約を確認すること。
- ハンドル・バッファの解放漏れ(設計上の割り切り)** — プロセス終了時の明示破棄経路がない。長時間サービス化する 改修を行う場合は解放処理の追加が必要。

5. **行列サイズ 0 のガード** — GEMMul8 ブリッジは $m, n, k \leq 0$ で即成功を返す(全 GEMM がブリッジ経由のため、この一箇所で守られる)。
6. **cuBLAS ワークスペース未設定** — GEMMul8 が推奨する `cublasSetWorkspace` (32 MiB) は どのハンドルにも設定されていない(性能改修の候補。`gemmul8.hpp` のパフォーマンスノート参照)。
7. **列優先規約** — cuBLAS は Fortran BLAS と同じ列優先。OpenMX の実対称行列は対称性を利用して 行優先フラット配列をそのまま渡す箇所がある(転置と等価なため。`Eigen_lapack.c:1570-1574`)。エルミート行列は明示的に列優先へ詰め替える(`EigenBand_lapack.c:709-714`)。

4. GEMMu8 の使い方

4.1 GEMMu8 とは — 原理と出自

GEMMu8(GEMMulate)は理化学研究所 計算科学研究センター(RIKEN R-CCS)が開発する「INT8/FP8 行列エンジンを用いた GEMM エミュレーション」ライブラリである(責任開発者: Yuki Uchino 氏、MIT ライセンス)。本実装が取り込んでいるのは **v3.0.4**(上流コミット `884a2875...` に固定)。上流リポジトリ: <https://github.com/RIKEN-RCCS/GEMMu8>。

原理は **Ozaki Scheme II**(整数モジュラー技法/中国剰余定理 CRT に基づく浮動小数点行列積エミュレーション)である。

1. **スケーリング+量子化**: FP64 行列 A, B を指数調整し、互いに素な法 $p = 256, 255, 253, 251, 247, 241, \dots$ (8 bit に収まる整数、最大 20 個。 `src/oz2/common/table.hpp:12-31`)ごとの剰余をとって `num_moduli` 枚の INT8 行列に変換する。
2. **誤差なし整数行列積**: 法ごとに INT8 GEMM を実行する。実体は cuBLAS の `cublasGemmEx(..., CUDA_R_8I, ..., CUBLAS_COMPUTE_32I, ...)`、すなわち **INT8 Tensor Core(IMMA)** である。整数演算なので丸め誤差が入らない。
3. **CRT 復元**: 法別の結果から中国剰余定理で整数積を復元し、スケーリングを戻して $\alpha AB + \beta C$ を得る。

`num_moduli` (法の個数)を増やすほど高精度・低速になる。手順が決定的であるため、**同一設定なら結果は bitwise に再現する**(上流 README が明記)。GEMMu8 は、INT8 整数行列エンジンで DGEMM を行う先行研究 ozIMMU(Ootomo, Ozaki, Yokota 2024)系の Ozaki Scheme I を発展させた Scheme II の実装という位置づけである。引用すべき文献は巻末の参考文献を参照。

4.2 統合アーキテクチャ — 2 ファイル方式

OpenMX 本体は NVHPC `mpicc -std=c11` でビルドされる C コードであり、C++ テンプレート API を直接呼べない。また上流のビルドシステムは「ルーチン × 型 × バックエンド」ごとに大量のオブジェクトを生成する。そこで本実装は次の 2 ファイルで統合している。

(a) `gemmul8_openmx.cu` — 単一翻訳単位での明示的テンプレート実体化

上流のテンプレート定義ヘッダ(`gemm_impl.hpp`、 `worksize_impl.hpp` と、上流では別オブジェクトに分かれるカーネル定義ヘッダ 7 本)を 1 つの翻訳単位に include し、OpenMX が必要とする **4 シンボルだけ**を明示的実体化する(67 行の小さなファイル)。

`gemmul8_openmx.cu:31-44`(double 版、`cuDoubleComplex` 版と `workSize` 2 種も同様)

```
template std::vector<double> gemm<double, Backend::INT8, double, double>(
    cublasHandle_t,
    cublasOperation_t, cublasOperation_t,
    size_t, size_t, size_t,
    const double *,
    const double *const, size_t,
    const double *const, size_t,
    const double *,
    double *const, size_t,
    int, bool,
    void *const,
    void *const,
    void *const,
    bool, bool, bool, bool);
```

実体化されるのは `gemm<double,INT8>`、`gemm<cuDoubleComplex,INT8>`、`workSize<false,INT8,Func::gemm>`、`workSize<true,INT8,Func::gemm>` の 4 本。このオブジェクトだけを `ar` で固めて `third_party/GEMMu8/lib/libgemmul8.a` を自

前生成する (上流の同名ライブラリとは中身が異なる)。この方式は上流の内部ヘッダ構成に依存するため、**上流コミットのピン留めが必須**である (Makefile のコメントに明記。§ 4.7)。

(b) gemmul8_bridge.cu — extern "C" ブリッジ

C 側から呼べる 3 関数を輸出する (プロトタイプは `openmx_common.h:4105-4112`)。

- `cublasStatus_t openmx_gemmul8Dgemm(handle, transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)` — **cublasDgemm と完全同型の**シングネチャ(alpha/beta・ld とも任意)
- `openmx_gemmul8Zgemm(...)` — 複素版(同型)
- `openmx_gemmul8ReleaseWorkspaces()` — 全ワークスペースの解放

ブリッジは (i) 環境変数の解釈、(ii) ワークスペースの照会・確保・キャッシュ、(iii) 確保できない場合の native cuBLAS へのフォールバック、(iv) `gemmul8::gemm` 呼び出し、を担う。

gemmul8_bridge.cu:296-316(Dgemm 本体。Zgemm も同構造)

```
if (gemmul8_disabled("OPENMX_GEMMUL8_DISABLE_D", "GEMMUL8_DISABLE_D")) {
    report.reason = "environment disable";
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}

cublasStatus_t status =
    ensure_workspace<false>(handle, static_cast<size_t>(m), static_cast<size_t>(n), static_cast<size_t>(k),
        num_moduli, &work, &report);
if (status == CUBLAS_STATUS_ALLOC_FAILED) {
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}
if (status != CUBLAS_STATUS_SUCCESS) {
    return status;
}

(void)gemmul8::gemm<double, gemmul8::Backend::INT8>(handle, gemmul8_transa, gemmul8_transb, ..., num_moduli, fastmode, work);
```

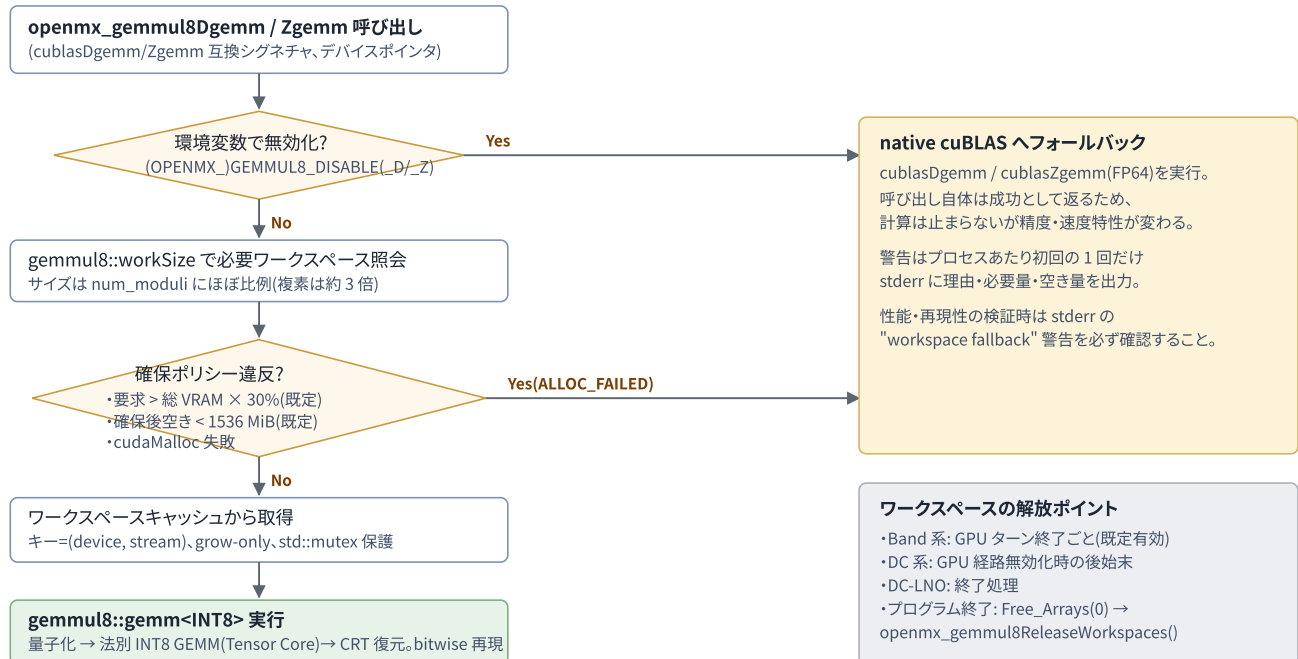


図 3: GEMMUL8 ブリッジのディスパッチフロー (gemmul8_bridge.cu)。

4.3 呼び出しパラメータと精度設定

パラメータ	既定値	環境変数(OPENMX_名が優先)	意味・挙動
<code>num_moduli</code>	15	<code>(OPENMX_)GEMMUL8_NUM_MOD_D / _Z</code>	法の個数(精度)。有効範囲 2~20、範囲外は黙って 15 に戻る。D/Z 独立に設定可
<code>fastmode</code>	false(accurate)	<code>(OPENMX_)GEMMUL8_FASTMODE_D / _Z</code> ("1" で有効)	高速スケーリング経路。精度が下がる
無効化	off	<code>(OPENMX_)GEMMUL8_DISABLE, _D / _Z</code>	native cuBLAS FP64 に固定
WS 上限比率	30%	<code>(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT</code>	要求が総 VRAM のこの割合を超えたら拒否(Band_DFT_NonCol は未設定時に 50 を setenv する)
確保後最低空き	1536 MiB	<code>(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB</code>	確保後の空き VRAM がこの値を割るなら拒否

- 実数(D)経路は `CUBLAS_OP_C` を `CUBLAS_OP_T` に静かに正規化する(実数では共役転置=転置)。
- `m, n, k ≤ 0` は何もせず成功を返す。
- OpenMX 側に GEMMUL8 の結果を FP64 直接計算と比較・検証するコードは存在しない。フォールバックはすべて メモリ/環境変数条件によるもので、精度条件によるものではない。
- skip-scaling 等の上流拡張機能は未使用。フェーズ別時間の戻り値も捨てられる。

4.4 ワークスペース管理

- 照会:** 呼び出しごとに `gemmul8::workSize<is_complex, INT8>(m, n, k, num_moduli)` で必要量を計算。
- キャッシュ:** `(device, cudaStream_t)` をキーとする `std::unordered_map` でプロセス全体共有、`std::mutex` 保護、grow-only(`gemmul8_bridge.cu:21-44, 207-241`)。cuBLAS ハンドルのストリームがキーになるため、**ハンドル(ストリーム)ごとに別領域**になる点に注意。
- サイズ感:** A 側 = num_moduli 枚の INT8 行列+シフトベクトル、B 側同様、C 側 = (num_moduli-1) 枚の中間行列 + 32 MiB の内部 BLAS 領域 + INT32 アキュムレータ。複素はさらに約 3 倍。 $n \approx 1$ 万規模で約 **1 GiB/ランク** (コード内コメント `Band_DFT_Col.c:285-288`)。
- 解放:** `openmx_gemmul8ReleaseWorkspaces()` のみ。呼び出し箇所は (1) `Free_Arrays(0)` (プログラム終了時、`Free_Arrays.c:24`)、(2) `Band_DFT_Col` の GPU ターン終了ごと(`OPENMX_BAND_GEMMUL8_TURN_RELEASE=0` でオプトアウト)、(3) DC 法の GPU 経路無効化時、(4) DC-LNO の終了処理。

4.5 複素数(Z)対応

OpenMX 側では複素行列を実行列に分解せず、`cuDoubleComplex` のまま `gemm<cuDoubleComplex, INT8>` に渡す。GEMMUL8 内部のネイティブ複素パスは、入力 1 個あたり低精度表現を 3 セット持ち、1 法あたり**実 INT8 GEMM 3 本**で複素積を構成する(4M 法ではなく 3M/Karatsuba 系。理論的背景は CRT に基づく複素行列積エミュレーションの論文、参考文献参照)。このため複素のワークスペースと演算量は実数の約 3 倍になる。

4.6 どの GEMM が GEMMUL8 で実行されるか

ブリッジ内に行列サイズによる振り分けは**ない**(GEMMUL8 が既定、フォールバックはメモリ/環境変数条件のみ)。コミット `66ce1e62` 以降は **GPU 上の全 GEMM がブリッジを経由する**(表 3-2)。代表例は `Band_DFT_Col` の一般化固有値問題変換 $H' = \tilde{U} + H\tilde{U}$ である(ここが最大の GEMM 負荷)。

`Band_DFT_Col.c:1452-1458` — 三重積変換(2 本目までを抜粋。逆変換 1 本が :1473 に続く)

```
wait_cudafunc(openmx_gemm8Zgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n, &alpha,
                                (cuDoubleComplex *)ctx->d_H, n, (cuDoubleComplex *)ctx->d_S, n, &beta,
                                (cuDoubleComplex *)ctx->d_tmp, n));

wait_cudafunc(openmx_gemm8Zgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, n, n, n, &alpha,
                                (cuDoubleComplex *)ctx->d_S, n, (cuDoubleComplex *)ctx->d_tmp, n, &beta,
                                (cuDoubleComplex *)ctx->d_H, n));
```

4.7 ビルド

- **取得:** `make` が git submodule ディレクトリで `git fetch --depth 1 origin $(GEMMUL8_COMMIT)` → `checkout` を実行し、コミット `884a2875...` に固定する (Makefile:716-722)。ピン留めの理由は Makefile コメントに明記: 「gemmul8_openmx.cu が GEMMUL8 内部を直接実体化するため、チェックアウトは内部レイアウトが一致するコミットに固定し続けなければならない」。
- **コンパイル:** NVHPC 同梱 CUDA 13.1 の `nvcc`、ホストコンパイラ `-ccbin nvcc++`、`-std=c++20 -O3 -diag-suppress 177 -DGPU_ARCH=$(arch) -gencode arch=compute_$(arch),code=sm_$(arch)`。アーキは `nvidia-smi --query-gpu=compute_cap` から自動検出 (本開発環境では `compute_120/sm_120`)。 `GEMMUL8_GPU_ARCH=90` のように手動指定も可能 (Makefile:161-167)。
- **環境サニタイズ:** `NVCC_GEMMUL8_ENV` が `CPATH` 等を空にして NVHPC のヘッダ・フラグ混入を遮断 (Makefile:170)。
- **リンク追加:** `-lcublasLt -lcuda -lnvidia-ml -lstdc++` (Makefile:172)。

4.8 GEMMUL8 使用上の注意点

1. **メモリ使用量** — ワークスペースは `num_moduli` にほぼ比例、複素は約 3 倍、 $n \approx 1$ 万で約 1 GiB/ランク。複数ランク共有 GPU では「30% 上限・1536 MiB 予約・ターン毎解放」の 3 段階で枯渇を防いでいる。既定値を変える場合はこの 3 つの整合を保つこと。
2. **静かな FP64 フォールバック** — ワークスペース拒否時は警告 1 回だけで cuBLAS FP64 に切り替わり、**速度と数値結果 (ビットパターン) の両方が変わる**。「なぜか遅い/結果が揺れる」ときは stderr の "GEMMUL8 workspace fallback" を確認する。決定的な再現が必要な検証では `OPENMX_GEMMUL8_DISABLE=1` で全無効化するか、メモリに十分な余裕を持たせる。
3. **精度設定** — 既定 15 moduli・accurate モード。範囲外の指定は黙って 15 に戻る。fastmode は高速だがスケーリング精度が落ちる。D と Z は独立に調整できる。
4. **初回呼び出しコスト** — 初回およびサイズ増加時に GiB 級の `cudaMalloc` が同期的に走る。ベンチマークは 2 回目以降の反復で評価すること。
5. **ビルドのハマりどころ** — (a) コミットピンを外すと内部ヘッダ構成の変更で `gemmul8_openmx.cu` が壊れる。(b) NVHPC 環境変数 (CPATH 等) を掃除しないと `nvcc` が NVHPC ヘッダを拾って失敗し得る。(c) `-lcublasLt`・`-lstdc++` のリンク忘れ。(d) アーキ自動検出は `nvidia-smi` が使える環境前提。(e) 単一アーキ `-gencode` のため別世代 GPU ではバイナリ非互換 (要再ビルド)。
6. **上流 API 追従** — 旧 API から現行構造 (884a287) への移行 (コミット `cb4d348d`) で `gemmul8_openmx.cu` は 149 行 → 67 行に簡素化された。ブリッジの公開シグネチャは不変で、C 側への影響はない。今後の上流更新時も「単一 TU が include する定義ヘッダの構成」を最初に確認するとよい。
7. **ハードウェア要件** — INT8 Tensor Core (IMMA) 必須。C++20 対応 `nvcc` (CUDA 12 系以降) が事実上必要 (検証済みは CUDA 13.1 のみ)。

5. cuSOLVER(gpusolver 層)の使い方

5.1 層の構成と命名の意図

固有値計算は cuSOLVER を **gpusolver** というベンダー中立名のラップ層経由で使う。層は次の 3 階層で構成される。

1. **汎用ラップ関数**(1 呼び出し完結型): `gpusolver_Syevd.c` / `gpusolver_Syevdx.c` の 5 関数。ホスト配列版と、OpenACC デバイス常駐配列を直接処理する `_openacc` 版がある(表 5-1)。冒頭 48 行のライセンスヘッダが示すとおり、NVIDIA CUDA Library Samples の `syevd/syevdx` サンプルを改変した出自を持つ。
2. **共通ユーティリティヘッダ**: `openmx_gpusolver_dense_utils.h` (79 行)。`OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` の 1-based/0-based 版 4 関数を static 定義し、`info != 0` なら `MPI_Abort` する。分極・DMmu・CWF・LNO 系の 12 ファイルが include する。
3. **ソルバ内常駐コンテキスト**: `BandColGpuSolverCtx`、`ClusterColGpuSolverCtx`、`DCGpuSolverCtx`、`DCLNO_GpuSolverCtx`、`Krylov` のスレッド別ワークスペースプールなど。ハンドル・デバイスバッファをキャッシュし、cuSOLVER API を直接呼ぶ(第 3 章と共通の構造)。

命名の意図: ベンダー中立化は「命名規約による抽象化」であり、マクロや関数ポインタによる切替機構はまだ存在しない。関数名・構造体メンバ名・入力キーワードを `gpusolver` に統一し(例: `cusolverDnHandle_t gpusolver;`)、cuSOLVER の型名・API 名は NVIDIA ビルドの実装詳細としてそのまま残す。AMD(hipSOLVER/rocSOLVER)対応は 同一の命名規則の下で別途実装する想定である。入力キーワードの旧名 `cusolver` は警告付きの非推奨エイリアスとして残る。開発初期に評価された MAGMA は削除済みで、現行ソースに参照はない(§ 2.1)。

関数(<code>gpusolver_Syevd(x).c</code>)	cuSOLVER API	データ所在	現在の呼び出し元
<code>gpusolver_Syevd(A,W,m)</code>	Xsyevd(全固有値)	ホスト⇄デバイス 転送内蔵	休眠 呼び出し元なし
<code>gpusolver_Syevdx(A,W,m,MaxN)</code>	Xsyevdx(下から MaxN 本)	同上	<code>Eigen_lapack.c:1576,1667</code> 、 <code>Cluster_DFT_Col.c:607,649</code> 、 <code>dense_utils</code> 経由多数
<code>gpusolver_Syevdx_openacc(...)</code>	Xsyevdx	OpenACC 常駐 (present + host_data)	<code>Eigen_lapack.c:1617,1645</code>
<code>gpusolver_Syevdx_Complex(...)</code>	Xsyevdx(CUDA_C_64F)	ホスト⇄デバイス 転送内蔵	<code>EigenBand_lapack.c:716</code> 、 <code>Band_DFT_Col.c:1419</code> 、 <code>dense_utils</code> 経由多数
<code>gpusolver_Syevdx_Complex_openacc(...)</code>	Xsyevdx(CUDA_C_64F)	OpenACC 常駐	<code>EigenBand_lapack.c:677,741</code>

表 5-1: 汎用ラップの公開 API

5.2 使用 API の詳細

- すべて **generic(X)API・64-bit 整数版** を使用: `cusolverDnXsyevd(_bufferSize)` / `cusolverDnXsyevdx(_bufferSize)`。legacy API(`cusolverDnDsyevd` 等)や `Zheevd` 系の別名 API は一切使わず、複素も Xsyevdx に `CUDA_C_64F` を渡す統一形。一般化固有値ソルバ `cusolverDnXsygvd` も使わない(§ 5.3)。
- 共通パラメータ: `params = NULL`、`jobz = CUSOLVER_EIG_MODE_VECTOR`、`uplo = CUBLAS_FILL_MODE_LOWER`、範囲指定は `CUSOLVER_EIG_RANGE_I` (`il=1, iu=MaxN`)。 `m == MaxN` のとき `RANGE_ALL` に切り替える実装と常に `RANGE_I` の実装が混在する。
- ハンドル: 汎用ラップは呼び出しごとに `cusolverDnCreate` → non-blocking ストリーム生成 → 計算 → 全解放。常駐コンテキストはキャッシュする(§ 3.2)。

- ワークスペース: 全経路で `_bufferSize` によりデバイス/ホスト両方の必要量を照会してから確保する。常駐側は grow-only キャッシュ。DC 法だけは確保前に `cudaMemGetInfo` による事前判定(preflight)を行い、不足なら GPU 経路を無効化して CPU で継続する(`Divide_Conquer.c:833-880`)。

gpusolver_Syevdx.c:103-117 — 中核の呼び出し形(64-bit generic API)

```
wait_cudafunc(cusolverDnXsyevdx_bufferSize(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu,
                                           1L, MaxN, &h_meig, CUDA_R_64F, d_W, CUDA_R_64F,
                                           &workspaceInBytesOnDevice, &workspaceInBytesOnHost));

wait_cudafunc(cudaMalloc((void **)&d_work), workspaceInBytesOnDevice));
h_work = (workspaceInBytesOnHost == 0) ? NULL : malloc(workspaceInBytesOnHost);
...
wait_cudafunc(cusolverDnXsyevdx(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu, 1L, MaxN,
                                &h_meig, CUDA_R_64F, d_W, CUDA_R_64F, d_work, workspaceInBytesOnDevice, h_work,
                                workspaceInBytesOnHost, d_info));
```

5.3 一般化固有値問題 $Hc = \epsilon Sc$ の処理

cuSOLVER の一般化ソルバは使わず、OpenMX 従来の **S** の固有分解によるレイディン(対称)直交化を各経路が GPU 上で自前実装する。Cholesky 分解は一切使わない。手順は共通して次の 6 ステップである(図 4 も参照)。

1. S を Xsyevdx(全固有値)で対角化 → 固有ベクトル U、固有値 λ
2. 小さい/負の固有値の処置(経路により異なる: Band/Cluster は 1×10^{-10} に切上げ、DC は `OLP_eigen_cut` 未満を切り捨て、DC-LNO は `fabs`)
3. U の列を $1/\sqrt{\lambda}$ でスケール(`cublasDdggmm/Zdggmm`) → $\tilde{U}$
4. $H' = \tilde{U}^\dagger H \tilde{U}$ を GEMM 2 回で構成(GEMM_{ul8}。表 3-2)
5. H' を Xsyevdx(il=1..MaxN)で対角化 → 固有値 ϵ 、固有ベクトル z
6. 逆変換 $c = \tilde{U}z$ を GEMM で実行

Cluster_DFT_Col.c:249-262 — S 対角化と $1/\sqrt{\lambda}$ スケーリング(GPU 上のレイディン直交化)

```
if (rebuild || !(ctx->transformed_s_valid && ctx->transformed_s_dim==n)){
    ClusterCol_GpuSolver_EigenDevice(ctx->d_S,n,n,ko0+1);

    for (int l=1; l<=n; l++){
        if (ko0[l]<1.0e-10) ko0[l] = 1.0e-10;
        ko0[l] = 1.0/sqrt(ko0[l]);
    }

    wait_cudafunc(cudaMemcpyAsync(ctx->d_W,ko0+1,sizeof(double)*(size_t)n,cudaMemcpyHostToDevice,ctx->stream));

    old_s = ctx->d_S;
    new_s = ctx->d_tmp;
    wait_cudafunc(cublasDdggmm(ctx->cublas,CUBLAS_SIDE_RIGHT,n,n,
                              old_s,n,ctx->d_W,1,new_s,n));
```

注意(数値的挙動の差): S の小さい固有値の処置(切上げ / 切捨て / `fabs`)が経路間で統一されていないため、悪条件の重なり行列を持つ系では、同じ物理系でもソルバ経路によって数値挙動が微妙に異なり得る。

部分固有値数 MaxN の決め方: Band(Col) は `MaxN = (TZ-charge)/2 + max(0.4*Ne/2, 100)` (`Band_DFT_Col.c:1813-1818`)、Cluster は SCF 第 1 反復で全数、以後 `(TZ-charge)/2 + max(0.1n~0.2n, 400)` (`Cluster_DFT_Col.c:357-379`)。占有帯+余裕分だけを解くことで syevdx の計算量を抑えている。

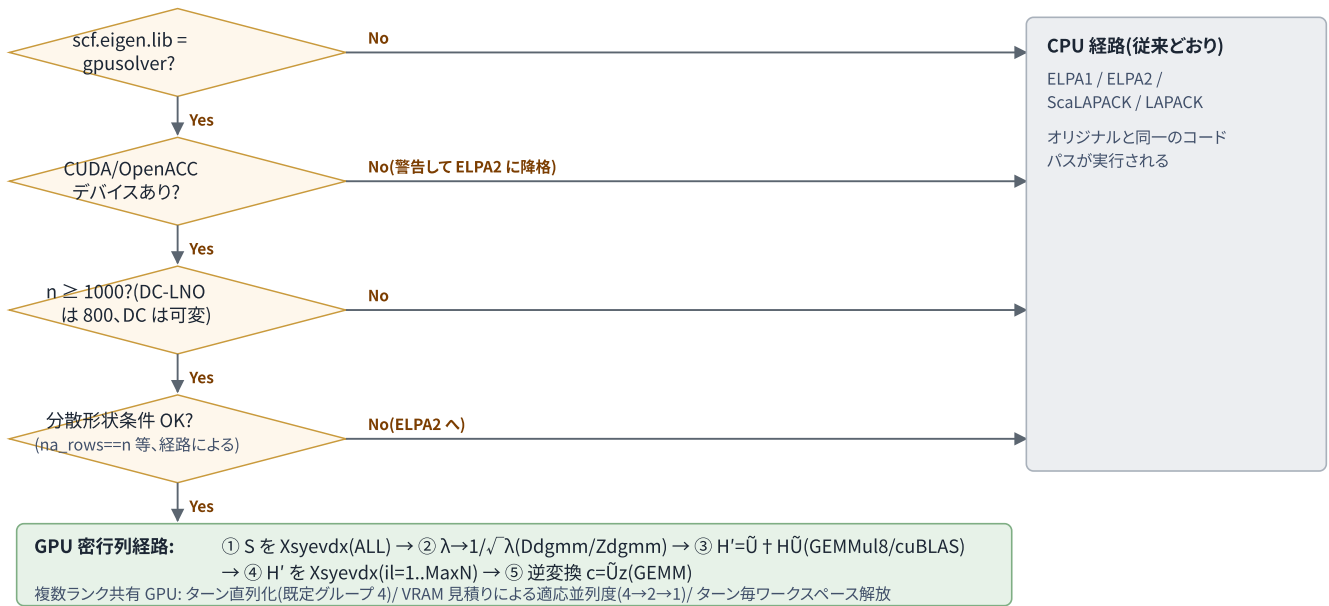


図 4: 固有値ソルバのディスパッチと GPU 密行列経路。

5.4 devInfo の検査と失敗時処理(経路別)

`d_info` は全経路でデバイスに確保し、計算後にホストへコピーして検査するが、**失敗時の方針は経路ごとに異なる**。改修時は次の表を前提にすること。

経路	info 検査	失敗時の挙動
汎用ラッパ(gpusolver_Syevdx 等)	呼び出し元に返すのみ	呼び出し元依存
dense_utils ヘッダ経由(11 ファイル)	あり	即 <code>MPI_Abort</code>
<code>Eigen_lapack.c:1590-1593</code>	info<0 のみ exit(10)	info>0(収束失敗)は素通り。ただし上位の <code>Eigen_lapack()</code> に固有ベクトル直交性検査つきリトライがあり、失敗時はソルバを CPU 系へ巡回して再計算(最大 3 回)
<code>EigenBand_lapack.c:25-32</code>	あり	CPU の Householder 法 <code>Eigen_HH</code> へフォールバック
Band_DFT_Col / Cluster_DFT_Col	あり(Band は h_meig も)	即 abort(exit / MPI_Abort)
Divide_Conquer(DC)	あり(h_meig も)	sticky 無効化フラグ を立てて以後そのランクは CPU の DC ソルバへ。GEMM8 ワークスペースも解放
Divide_Conquer_LNO	あり(h_meig は未検査)	exit(10)(フォールバックなし)
Krylov	あり(h_meig も)	CPU LAPACK(<code>Eigen_lapack2</code>)へその場でフォールバック

表 5-2: devInfo 検査と失敗時処理の経路差

5.5 並列度制御 — GPU ターン機構と適応制限

5.5.1 Band(Col): GPU ターンの直列化

複数ランク(スピン世界 × k 点世界の代表ランク)が同一 GPU に同時に密ソルバを投げると VRAM が枯渇し、GEMM8 が FP64 フォールバックに落ちて全体が遅くなる。そこで「スピン × k 点」の大域通し番号 **GPU ターン**を導入し、既定で直列化する(コード内コメント: 8 オーナーランク/1 GPU では並行投入より約 27% 高速)。

`Band_DFT_Col.c:2637-2647` — ターン直列化の骨格

```

my_gpu_turn = spin * T_knum + kloop;
const int serialize_gpu_turns = BandCol_SerializeGpuSolverGpuTurns();
const int gpu_turn_group = serialize_gpu_turns ? BandCol_GpuTurnGroup() : 1;
const int first_gpu_turn = serialize_gpu_turns ? 0 : my_gpu_turn;
const int last_gpu_turn = serialize_gpu_turns ? Num_Comm_World1 * T_knum : (my_gpu_turn + 1);
for (int gpu_turn = first_gpu_turn; gpu_turn < last_gpu_turn; gpu_turn++) {
    if (serialize_gpu_turns && gpu_turn % gpu_turn_group == 0) {
        MPI_Barrier(mpi_comm_level1);
    }

    if (owns_global_dense_rank && my_gpu_turn == gpu_turn) {

```

同時に走ってよいターン数(グループ幅)は `OPENMX_BAND_GPU_TURN_GROUP` → `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE` → 既定 4 の優先順位で決まる。`all_knum != 1` の第 1 対角化ループは別ノブ `OPENMX_BAND_GPU_MAX_CONCURRENT_K` (既定 4) で制御される。後者はコミット `8c7801fa` (Limit band GPU k-turn concurrency) で **上限 4 にクランプ**され、それ以上を指定しても 4 に丸められる。

5.5.2 Band(NonCol): VRAM 見積りに基づく適応制限

コミット `43e30c65` (Limit band GPU solver concurrency adaptively) で、非共線バンドは固定値に代わり「必要デバイスメモリの明示的見積り × 実測空き容量」で並列度を決めるようになった。

1. **所要量モデル**(`Band_DFT_NonCol.c:767-817`): overlap / Hamiltonian / 波動関数の 3 フェーズのピークを見積る。syevdx ワークスペースは実際に `_bufferSize` を照会して算入(照会失敗時は $4 \cdot n^2 \cdot 16$ バイトで代用)。
2. **判定**: 「1 ターン所要 × 同時ランク数 + 予約(OpenACC/GEMM_u8 スクラッチ用に **まる 1 ターン分**のクッション)」が 空き VRAM に収まるかを検査。
3. **決定**: 収まらなければ 4→2→1 に段階縮小し、同一 GPU 共有ランクのコミュニケーター (`MPI_Comm_split_type(SHARED)` + デバイス ID) で `MPI_Allreduce(MIN)` を取り、**全ランク一致の限界値**に揃える。

さらにコミット `8c7801fa` で、`OPENMX_BAND_GPU_MAX_CONCURRENT_K / MAX_RANKS_PER_DEVICE` の**明示指定値(上限 4)にもこの適応制限が適用**されるようになった (`BandNonCol_AutoGpuTurnLimit` を必ず経由)。環境変数で大きな値を与えても、VRAM に収まらなければ自動縮小される。

`Band_DFT_NonCol.c:847-859` — 判定部(コメント原文つき)

```

required = required_bytes*(size_t)concurrent_ranks;
reserve = BandNonCol_RootDenseReserveBytes(total_bytes,required);

/* Dense non-collinear turns also use OpenACC and GEMMu8 scratch space
   around the explicitly estimated matrices. Keep a full extra turn-sized
   cushion so the automatic limiter does not choose a group that only
   fits on paper and then fails in cuMemAlloc. */
if (reserve<required) reserve = required;

if (group_required!=NULL) *group_required = required;
if (reserve_bytes!=NULL) *reserve_bytes = reserve;

return (required<=free_bytes && reserve<=free_bytes-required);

```

5.5.3 DC-LNO: direct per-rank dispatch

コミット `f508a2a6` (Enable direct GPUSOLVER dispatch for DC-LNO) で、DC-LNO は「GPU オーナーランクが他ランクの固有値問題を MPI 経由で代行する**プロキシ方式**」から、「**全ランクが自分の局所問題(次元 ≥ 800)を直接 GPU に投入する方式**」に転換された。プロキシ機構は `DCLNO_ENABLE_SHARED_GPU_PROXY 0` (`Divide_Conquer_LNO.c:82`) で実験用に温存されている。ノード内で使う GPU 台数は入力キーワード `scf.Gpu.Num` で上限化できる。

Divide_Conquer_LNO.c:509-529(抜粋) — S 対角化 → スケール → GEMM₈ 変換 → H 対角化の一括 GPU 実行

```

DCLNO_GpuSolver_Eigen(ctx->d_S, NUM, NUM, ko + 1);

for (l = 1; l <= NUM; l++) {
    ko[l] = 1.0 / sqrt(fabs(ko[l]));
}
wait_cudafunc(cudaMemcpyAsync(ctx->d_W, ko + 1, ..., cudaMemcpyHostToDevice, ctx->stream));

wait_cudafunc(cublasDdgm(ctx->cublas, CUBLAS_SIDE_RIGHT, NUM, NUM,
    ctx->d_S, NUM, ctx->d_W, 1, ctx->d_tmp, NUM));
wait_cudafunc(openmx_gemm8Dgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, NUM, NUM, NUM, &alpha, ctx->d_H, NUM,
    ctx->d_tmp, NUM, &beta, ctx->d_S, NUM));
wait_cudafunc(openmx_gemm8Dgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, NUM, NUM, NUM, &alpha, ctx->d_tmp, NUM,
    ctx->d_S, NUM, &beta, ctx->d_H, NUM));

DCLNO_GpuSolver_Eigen(ctx->d_H, NUM, NUM2, ko + 1);

```

5.6 MPI ランク → GPU デバイス割当

- **collective 版**(set_cuda_default_device_from_local_rank.c:34-56): MPI_Comm_split_type(MPI_COMM_TYPE_SHARED) でノード内 local rank を求め、local_rank % deviceCount を cudaSetDevice 。全ランクが揃って呼ぶ必要がある(片側だけ呼ぶとデッドロック)。
- **noncollective 版**(:58-73): SCF 内ホットパス用。環境変数 OMPI_COMM_WORLD_LOCAL_RANK → MV2_COMM_WORLD_LOCAL_RANK → SLURM_LOCALID → PMI_LOCAL_RANK の順で local rank を取得。どれも無ければ大域 rank で代用(この場合マルチノードでは割当が偏り得る)。
- **OpenACC 側も対で設定** (set_openacc_device_from_local_rank.c): 同じ規則で acc_set_device_num 。CUDA と OpenACC の両ランタイムに同一デバイスを設定することが必須という設計。

5.7 LU_Zinverse_GPU — NEGF 向けの休眠実装

休眠コード: 複素 LU 逆行列の GPU 版 LU_Zinverse_GPU (Lapack_LU_inverse.c:347-444)は、cuSOLVER ではなく **cuBLAS のバッチ API** cublasZgetrfBatched / cublasZgetriBatched (batch=1)で実装されている。A は OpenACC deviceptr、特異行列検出時は abort、結果は acc kernels で書き戻す。しかし公開関数 Lapack_LU_Zinverse からの呼び出しは /* LU_Zinverse_GPU(n, A, handle); */ とコメントアウトされており(:63)、既定は従来の CPU 実装 (LAPACK zgetrf/zgetri)のまま。呼び出し元は NEGF の表面グリーン関数計算 (TRAN_Calc_SurfGreen.c ほか)であり、**将来 NEGF 系の複素逆行列を GPU 化するための布石**である。

5.8 cuSOLVER(gpusolver 層)使用上の注意点

1. **ワークスペース量の見積り** — $n \times n$ 複素行列 1 本で $16n^2$ バイト、ソルバ本体+ワークスペース+GEMM スクラッチでその数倍が必要になる。コード内コメントの設計前提は「16 GB 級 GPU を 8 ランクで共有」。VRAM に対して系が大きい場合は並列度ノブ (§ 5.5、第 10 章)を先に調整する。
2. **wait_cudafunc の無限ループリスク** — § 3.7 と同じ。恒久的エラーではハングする。
3. **devInfo 検査の不均一** — 表 5-2 のとおり、収束失敗からの回復可能性が経路により異なる。Band/Cluster/DC-LNO/dense_utils 系は「失敗=即終了」である。
4. **CPU フォールバックの有無** — 実行時フォールバックを持つのは DC(sticky 無効化)、Krylov、EigenBand_lapack(→Eigen_HH)、Eigen_lapack(リトライ巡回)のみ。デバイス不在時のみ全体が ELPA2 に降格する。

5. **対称性と 0/1-based の変換** — 全呼び出しが uplo=LOWER の対称/エルミート前提。固有値配列の 0/1-based ずらし (`W[l]=W[l-1]`) が各所で必要で、dense_utils が吸収している。
6. **S の条件数処理が経路で不統一** — § 5.3 の警告参照。
7. **再現性** — 変換 GEMM は GEMMu8 で実行されるため丸め挙動が cuBLAS FP64 と異なり、メモリ状況による フォールバックで実行ごとに経路が変わり得る(対策は § 4.8-2)。
8. **all_knum≠1 の Band GPU 経路**は `na_rows==n && na_cols==n` を要求し、満たさないと abort する。分極・DMmu 系は同条件を満たさない場合、黙って ELPA2 に落ちる。

6. OpenACC と生 CUDA の使い分け

6.1 全体像

ループ並列のオフロードは NVHPC の OpenACC(`-acc -Minfo=accel`)で記述されている。要点:

- **GPU 専用の `#ifdef` ガードは存在しない。**CUDA ヘッダ(`cublas_v2.h` 等)は `openmx_common.h:5-7` で無条件 include されるため、**このツリーは CPU 専用にはビルドできない** (CPU 用はオリジナルまたは `makefile.cpu_intel.bak` を使う)。ゲートはすべて実行時フラグである。
- `-gpu=ccXX` 指定はなく、ターゲット CC はビルドマシンの GPU 自動検出に依存する。**managed memory は不使用。**
`async` 節・`acc_get_cuda_stream` も全コードで不使用。
- `#pragma acc` を持つファイルは約 26。最厚は `Band_DFT_NonCol.c` (63 箇所)、`Divide_Conquer.c` (56)、`Cluster_DFT_NonCol.c` (48)、`Band_DFT_Col.c` (46)、`Force.c` (40)。**Krylov.c は OpenACC ゼロ**(`-acc` なしでコンパイルされ、生 CUDA のみ)。
- 基本形は `parallel loop gang` + `loop vector reduction`。密度行列構築など再現性が重要な縮約は `loop seq` (逐次)で加算順序を CPU と一致させる。

6.2 データ常駐の 3 パターン

1. 呼び出し毎の一時 data 領域(`copyin/copyout`) — `Set_*`、`Force`、`Total_Energy_GPU`。OpenMX の多次元ポインタ配列はデバイスで辿れないため、**必ずホスト側で 1 次元フラット配列に詰め替えてから転送する**設計で統一されている。
2. `enter data / exit data + update` による SCF スコア常駐 — 非共線系ソルバが中心。例: `Cluster_DFT_NonCol.c` は S を `enter data create` → デバイス上で対角化・スケール → `update self` → `exit data delete`。固有ベクトルは DM 構築まで常駐させる。
3. 生 `cudaMalloc` + `deviceptr` ブリッジ — `Band_DFT_Col.c` / `Cluster_DFT_Col.c` は static コンテキストの `d_H/d_S` (`cudaMalloc` 領域)に対し、OpenACC 側から `#pragma acc parallel loop deviceptr(d_H) + acc atomic update` で疎→密の scatter-add を実行する。OpenACC 管理メモリと `cudaMalloc` メモリが 1 カーネル内で混在する唯一のパターン。

ライブラリへのポインタ受け渡しは `host_data use_device` (全 23 箇所)で行う。`present` で常駐を表明したうえで、ブロック内では配列名がデバイスアドレスに置換され、`cuSOLVER/cuBLAS/GEMMv8` にそのまま渡せる(H2D/D2H 転送なし)。

`gpsolver_Syevdx.c:143-151` — OpenACC 常駐配列を `cuSOLVER` に直接渡す典型形

```
#pragma acc data      present(A[0 : m * m])
#pragma acc data      present(W[0 : MaxN])
#pragma acc host_data use_device(A, W)
{
    cusolverDnHandle_t cusolverH = NULL;
    wait_cudafunc(cusolverDnCreate(&cusolverH));
    cudaStream_t stream = NULL;
    wait_cudafunc(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
    wait_cudafunc(cusolverDnSetStream(cusolverH, stream));
```

注意(present エラー): `_openacc` 系ラッパは呼び出し側が data 領域を確立していることを前提とする。確立せずに呼ぶと NVHPC ランタイムの FATAL(present table lookup failed)で落ちる。現行コードは「呼び出し側が必ず `enter data` を先行させる」規約で統一されている。

6.3 同期の設計

OpenACC の既定キューとライブラリ用 CUDA ストリームは接続されていない。順序保証は (1) ライブラリ呼び出し直前の `#pragma acc wait`、(2) ライブラリ用ストリームの `cudaStreamSynchronize`、(3) 同期 `cudaMemcpy` の明示同期のみで行う。性能よりも正しさと単純さを優先した設計である。

6.4 openmx_cuda_kernels.cu — 検証済み・未結線の生 CUDA カーネル集

重要(現状の正確な位置づけ): `openmx_cuda_kernels.cu` / `.h` (696+103 行)は、`Cluster_DFT_NonCol.c` / `Set_Hamiltonian.c` / `Set_OLP_Kin.c` の OpenACC 計算領域を生 CUDA カーネルで書き直したモジュールである。ただし 2026-07-04 時点では **git 未追跡・Makefile のビルド対象外・.c 側からの呼び出し 0 件**であり、**現行バイナリでは対応する OpenACC 版が実行される**。bitwise 一致の検証は完了しているが結線はされていない(将来の置き換え候補)。

設計方針はファイル冒頭コメントに明記されている(原文): "every reduction that was sequential in the OpenACC code is kept sequential per thread here, so the floating point summation order is unchanged; $1.0/\sqrt{x}$ is used instead of the approximate `rsqrt` intrinsic" — すなわち**逐次縮約の順序保存**と**`rsqrt` 近似命令の回避**による CPU との bitwise 再現である。主なカーネル:

カーネル / ラップ	計算内容	縮約設計
<code>dense_scatter_add_kernel</code>	疎→密の scatter-add(0 詰め+加算)	<code>atomicAdd</code> (順序非決定)
<code>scale_columns_invsqrt_kernel</code>	$S^{-1/2}$ 構築の列スケーリング	縮約なし。 $1.0/\sqrt{w}$ (<code>rsqrt</code> 不使用)
<code>build_ss2_kernel</code> / <code>assemble_hs2_kernel</code>	非共線 $2n \times 2n$ の Ss2/Hs2 組立	縮約なし
<code>calc_dm_noncol_kernel</code> <CALC_EDM>	密固有ベクトルから DM/EDM 10 成分	状態 k ループを 1 スレッド逐次 (旧 OpenACC loop seq と同順)
<code>setham_base_kernel</code> / <code>setham_me_kernel</code>	H の base 結合 / 格子積分行列要素	格子点ループを 1 スレッド逐次
<code>olpkin_radial_kernel</code> + テーブル常駐ヘルパ	動径 k 積分(重なり・運動行列要素)	k 格子ループを 1 スレッド逐次。動径テーブルをプロセス寿命でデバイス常駐

共通基盤として grow-on-demand の `DeviceBuffer` キャッシュ、`block=256` 固定、全ラップ末尾の `cudaGetLastError` + `cudaDeviceSynchronize` (他ストリームの cuSOLVER/cuBLAS と自由に混在可能、とヘッダに明記)を持つ。なお現行の OpenACC 版 `Set_OLP_Kin.c:601` 等は `loop vector reduction` (並列縮約)であり、この CUDA 版の逐次縮約とは加算順序の規約が異なる。

6.5 NVHPC 固有の注意 — ミスコンパイル回避の実例

NVHPC 26.3 の OpenACC(`-acc`)には特定コードを壊す既知の不具合があり、Makefile は**ファイル単位で `-acc` を外す**ことで回避している(理由コメント付き)。これは NVHPC のバージョンを変える際に必ず再検証すべきポイントである。

ファイル	コンパイル指定	Makefile コメントの理由(要約)
<code>Occupation_Number_LDA_U.c</code> / <code>Eff_Hub_Pot.c</code> / <code>Total_Energy.c</code>	<code>CC_NOACC</code> (-O1, -acc なし)	NVHPC 26.3 + -acc が LDA+U Type2/cFLL の占有数・ポテンシャル経路をミスコンパイルする
<code>zero_cfrac.c</code>	<code>CC_NOACC</code>	-acc が DSYGV を失敗させ、ON2 法の極を壊す
<code>RestartFileDFT.c</code>	<code>CC_NOACC</code>	過分割 MPI 実行のリスタート I/O で segfault し得る
<code>MD_pac.c</code>	<code>CC_NOACC</code>	SCF 後の MD 制御経路で segfault し得る
<code>Krylov.c</code>	<code>CC_NOACC_03</code> (-O3, -acc なし)	GPU 化は生 CUDA API のみで行う(OpenACC 不使用)

ファイル	コンパイル指定	Makefile コメントの理由(要約)
Band_DFT_NonCol.c / Divide_Conquer.c	CC3 (-O2)	最適化レベル調整
truncation.c / OutData.c	CC2 (-O1)	同上
DFTD3vdW_init.c	CC_DFTD3 (-O0)	同上

表 6-1: ファイル別コンパイル調整(NVHPC 26.3 不具合回避を含む)

このほか、`Total_Energy.c` を `-acc` なしでビルドする必要から、OpenACC カーネル部分だけを別翻訳単位 `Total_Energy_GPU.c` (`-acc` 付きでビルド)に分離するという構成が生まれている (§ 7.8)。また `Force.c` には旧 NVHPC の OpenMP バグ回避として `#if NVHPC_VERSION >= 250000` の条件付きで OpenMP 並列を再有効化する箇所がある。`Divide_Conquer.c:3695` の `if (use_dc_openacc)` ガードには「// compiler's bug」というコメントが残る。

7. 計算経路別の GPU 実装詳細

7.1 バンド計算(コリニア) — Band_DFT_Col.c

最大規模の改修対象(実質約 3,700 行の変更)。GPU 密行列経路の条件は `scf_eigen_lib_flag==GPUSOLVER && n ≥ 1000` (+分散形状条件)。特徴:

- **密行列構築キャッシュ:** 疎行列(H, S)から k 点ごとの密行列を組み立てる部分を OpenACC 化し、ペア構造の「fingerprint」付きキャッシュ(`entries / phase_r / phase_i`)を `enter data` で常駐させる。位相のみ `update device` で更新。scatter-add は `deviceptr + acc atomic update (Band_DFT_Col.c:1539-1622)`。
- **Set_Hamiltonian の packed H/S キャッシュを利用:** 対角化前の行列集約を、Set_Hamiltonian が構築した packed 形式 (§ 7.7) から直接行う。
- **対角化と変換:** S 変換行列($\tilde{U}$)は初回に計算してホスト側 `transformed_s` に退避し、SCF 反復間で再利用 (`Band_DFT_Col.c:1138-1177, 2649-2684`)。H' の構成は GEMM_{ul8} Zgemm × 3 (§ 4.6)、対角化は `cusolverDnXsyevdx`。固有ベクトルは DM フェーズまでデバイス常駐(コメント:2708)。
- **all_knum != 1 経路:** メモリ削減のため S を毎ターン再対角化する設計(オリジナル譲りの意図的なメモリ/計算のトレードオフ)。この経路は `na_rows==n && na_cols==n` を要求する。
- **GPU ターン直列化と VRAM 対策:** § 5.5.1、§ 2.5 のとおり。SCF 反復間のデバイスバッファ常駐は既定 OFF、ONにしても空き 2048 MiB 未満なら自動オフ。ターンごとに GEMM_{ul8} ワークスペースを解放。
- **DM フェーズ:** OpenACC の `gang + loop seq` 縮約で、加算順序を CPU と一致させる (`Band_DFT_Col.c:967-1001`)。

7.2 バンド計算(非コリニア) — Band_DFT_NonCol.c

- OpenACC 常駐型(`enter data` 中心)の root-dense 経路。位相付き密行列構築(scatter-add)、S 対角化+S^{-1/2}、2n×2n スピノル H2/S2 の組立、U†HU(GEMM_{ul8} Zgemm × 2)、固有ベクトル逆変換、DM/EDM(`loop seq`)までを GPU で実行。
- GEMM は `66ce1e62` 以降 GEMM_{ul8}(表 3-2)。OpenACC キューとの順序は `#pragma acc wait` → `openmx_gemmul8Zgemm` → `cudaStreamSynchronize` で明示制御 (`Band_DFT_NonCol.c:433-455`)。
- **適応並列度制限 (§ 5.5.2)**と、複数 k 世界の同時実行可否判定 (所要合計の Allreduce SUM vs 最小空きの Allreduce MIN。収まらなければ「Serializing non-collinear dense GpuSolver k-worlds」と表示して直列化)を持つ。
- GPU メモリ見積りのため一時ハンドルで `Xsyevdx_bufferSize` だけを照会する専用関数を持つ(:729-765)。

7.3 クラスタ計算 — Cluster_DFT_Col.c / Cluster_DFT_NonCol.c

- **Col:** `ClusterCol_GpuSolverRootDensePath` (:448-545)が本経路。スピン世界の rank 0 だけが密行列を集約して GPU で解き(S 対角化 → Ddgmm → GEMM_{ul8} Dgemm × 2 → Xsyevdx → 逆変換も GEMM_{ul8} Dgemm。`66ce1e62` で cublasDs_{ym}mm+Dgemm から切替)、固有ベクトルを MPI_Send/Recv で分配する。S 変換行列はデバイス上にキャッシュ。旧実装 `ClusterCol_GpuSolverDensePath` (S 対角化のみ GPU、GEMM は CPU dgemm)はコード上残るが到達不能。
- **NonCol:** root-dense 対角化 (実 Dsyevx + 複素 Zheevx)。U†HU は GEMM_{ul8} D/Zgemm 計 12 回 (`Cluster_DFT_NonCol.c:1355-1393`)。密固有ベクトル `dense_evec` を `enter data create` で常駐キャッシュし、後段へは `DFT.c` から `Cluster_DFT_NonCol_ScatterGpuSolverCachedEvec` を呼んで配布する。DM/EDM は `loop seq` 縮約。
- MaxN(部分固有値数)は SCF 第 1 反復のみ全数、以後は占有数+マージン (§ 5.3)。

7.4 DC 法 — Divide_Conquer.c

- 原子ごとの部分クラスタ行列(局所次元 NUM)を各ランクが独立に処理する。GPU 化条件は `NUM ≥ DC_GPU_Threshold()` (既定 1000、`OPENMX_DC_GPU_THRESHOLD` で変更可)。
- Col 側は生 CUDA 実装:** `DCGpuSolverCtx` に `cudaMalloc` バッファ+pinned ホストバッファ (`cudaMallocHost` 、本実装唯一の使用箇所)。S 対角化(SCF 2 反復目まで)→ H' 構成 (`DC_GpuSolver_TryGpuDgemm` : GEMMmul8 → 失敗時 `cublasDgemm` → それも失敗なら CPU)→ 活性部分空間を `cudaMemcpy2DAsync` でデバイス内切り出し → `Xsyevdx`。
- 多層の安全装置:** 確保前の VRAM preflight(`GPUSOLVER_MIN_FREE_AFTER_MB` 既定 1536 MiB)、ホスト malloc 失敗検出、固有値分解失敗検出 — いずれかで **sticky 無効化フラグ**を立て、以後そのランクは CPU の DC ソルバで継続する (GEMMmul8 ワークスペースも解放)。GPU 化された経路の中で最も防御的な実装である。
- NonCol 側は OpenACC 実装:** 部分空間変換(S 転置、 $H \cdot U \cdot M1$ 、 $M1 \cdot U + HU \cdot M1$ 、逆変換)を `kernels loop independent collapse` で実行し、対角化は `Eigen_gpusolver_x_complex_openacc` 経由。Col 側の旧 OpenACC 領域は `use_dc_openacc = 0` でハード無効化されている(コメント「compiler's bug」)。

7.5 DC-LNO 法 — Divide_Conquer_LNO.c

- 現行は **direct per-rank dispatch**(§ 5.5.3): 全ランクが `cudaSetDevice` を実行し、自分の局所問題($NUM \geq 800 = GPU_CPU_SWITCH_NUM2$)を直接 GPU に投入する。ワークスペースは `Max_Msize` 分をランクごとに事前確保。
- 旧方式の **GPU プロキシ機構**(同一 GPU 共有ランク群の rank 0 が他ランクの対角化を MPI タグ 41001-41004 で代行)は `DCLNO_ENABLE_SHARED_GPU_PROXY 0` で温存。
- ノード内 GPU 台数は `scf.Gpu.Num` (既定 30 ≡ 実質無制限)で上限化。同一 GPU 共有ランク群の コミュニケータ `DCLNO_gpu_group_comm` を構築する。
- 失敗時のフォールバックはなく `exit(10)`。`h_meig` は未検査(表 5-2)。

7.6 Krylov 法 — Krylov.c

- OpenACC 不使用・生 CUDA のみ** (`-acc` なしの `CC_NOACC_03` でコンパイル)。OpenMP スレッドごとの `Krylov_GPU_Workspace` プール(`d_A/d_B/d_C/d_W/d_work` + 専用 non-blocking ストリーム)を持つ。
- `Krylov_Dgemm` (:311): H2D Async → `openmx_gemm18Dgemm` → D2H → `StreamSynchronize` (`66ce1e62` で `cublasDgemm` から切替)。FLOP 数 $m \cdot n \cdot k < 5 \times 10^7$ は CPU BLAS(`KRYLOV_GPU_DGEMM_MIN_FLOPS`)。
- `Krylov_Eigen2` (:354): 埋め込みハミルトニアン標準固有値問題を $n == EVmax$ なら `Xsyevd`、部分なら `Xsyevdx` で解く。 $n < 1000$ は CPU。失敗時は `Eigen_lapack2` (CPU)にその場でフォールバック。
- S 変換は Krylov 基底構成(CPU の `Inverse_S_by_Cholesky`)で処理済みのため、GPU 側は標準固有値問題のみを扱う。

7.7 行列要素構築 — Set_Hamiltonian.c / Set_Nonlocal.c / Set_OLP_Kin.c

Set_Hamiltonian.c(実質約 1,800 行の変更)

- 格子積分の OpenACC 版は実装済みだが恒久無効**(`Set_Hamiltonian_MatrixElements_OpenACC_Enabled()` が `return 0`)。理由はコメントに明記: パッキングと転送のコストが支配的で CPU より遅い。base 結合($H = F_Kin \cdot H0 + F_VNA \cdot HVNA + F_NL \cdot HNL + \dots$)の GPU 版も「実測 10 ms vs CPU 2 ms」のため `OPENMX_SETHAM_BASE_GPU=1` のオプティン。
- 本命は packed H/S キャッシュ:** GPUSOLVER 用に H と S を 1 次元 packed 形式で保持する API 群を新設 (`Set_Hamiltonian_Build_GpuSolver_HS_Cache` ほか、宣言 `openmx_common.h:3339-3351`)。Band/Cluster ソルバが対角化前の密行列組立にこのキャッシュを直接使うことで、GPU 経路の転送・組立を高速化する。 `DFT.c:207-214` から SCF ごとに準備される。

- **メモリ安全装置:** `Set_Hamiltonian_DeviceMemoryOK` (:164-268) が必要量を見積り、同一 GPU 共有ランク群で Allreduce(必要量 SUM vs 空き MIN)して不足なら警告つきで CPU パスへ。OpenACC 実行は world 代表ランクのみに選抜される(`DFT.c`:159-204)。

`Set_Hamiltonian.c`:1409-1424(抜粋) — 格子積分の逐次縮約(実装済み・現在は無効の OpenACC 版)

```
#pragma acc loop vector
for (e = 0; e < (size_t)spin_count * mat_size; e++) {
    ...
    double sum = hbuf[hidx];
    int Nog;
    #pragma acc loop seq
    for (Nog = 0; Nog < NOLG; Nog++) {
        sum += vpotbuf[...] * orbs0buf[...] * orbs1buf[...];
    }
    hbuf[hidx] = sum;
}
```

`Set_Nonlocal.c`(+1,356/-306)

- 非局所射影行列要素 $\langle \phi | P \rangle$ の動径 k 積分を OpenACC 化(`collapse(2)` + `loop vector reduction`)。GPUSOLVER 時は常時有効。ベッセル関数テーブルは species 変化時のみ再計算。
- あわせて、固定サイズのスタック配列をオーバーフロー検査付きのヒープ確保に置き換える堅牢化も実施(`SetNonlocal_MallocArray` 等)。

`Set_OLP_Kin.c`(+529/-181)と `Spherical_Bessel.c`

- 重なり・運動エネルギー行列要素の動径積分オフロードは、データ転送律速で CPU より遅い実測 (コメント: 約 1.2 s vs 0.3 s)のため `OPENMX_OLPKIN_RADIAL_GPU=1` の **オプション**。さらにスレッド数 1(`Nthrds==1`)を要求する。
- CPU 側の実質的な高速化として、ペア距離 r の **bit パターン完全一致をキーとする球ベッセル関数テーブルキャッシュ**を追加(結果は bitwise 同一、シングルスレッド時のみ)。`Spherical_Bessel.c` もホットパスの 毎呼び出し malloc/free を除去した。

7.8 全エネルギー — `Total_Energy.c` + `Total_Energy_GPU.c`

ファイル分離の理由: NVHPC 26.3 の `-acc` が LDA+U 経路をミスコンパイルするため、`Total_Energy.c` 本体は `-acc` なし(`CC_NOACC`)でビルドせざるを得ない。そこで OpenACC カーネルだけを別翻訳単位 `Total_Energy_GPU.c` (`-acc` 付き、517 行)に分離した。ゲートは `TotalEnergyUseOpenACC()` : 非共線(SpinP_switch==3)では無効、それ以外は GPUSOLVER フラグに従う。

GPU 化された 4 種の格子縮約(いずれも毎回 copyin の一時 data 領域方式):

1. **EXC/EH1 格子積分**(Ena•Eef•EH1•EXC を単一 `parallel loop reduction(+5変数)` で縮約)
2. **双極子モーメント 6 成分**(格子座標は GN→(n1,n2,n3) をデバイス内で再構成)
3. **CWF 二重計上補正**(dcEH1/dcEXC)
4. **EH0 二中心項のバッチ計算** — 全原子ペアを一括配列化し、species ごとの球面格子・VPS メッシュ・V_H 原子テーブルをフラット化して転送。`gang` (ペア)× `vector reduction` (格子点)でエネルギーと r 微分(力 #6/#7 相当)を同時計算。スプライン補間は `#pragma acc routine seq` のデバイス関数として実装。

`Total_Energy_GPU.c`:39-47 — 格子縮約の典型形

```
#pragma acc data copyin(Density_Grid_B0[0:grid_count], Density_Grid_B1[0:grid_count], \
                        ADensity_Grid_B0[0:grid_count], PCCDensity_Grid_B0[0:grid_count], \
                        PCCDensity_Grid_B1[0:grid_count], dVHart_Grid_B[0:grid_count], \
                        Vxc_Grid_B0[0:grid_count], Vxc_Grid_B1[0:grid_count], \
                        RefVxc_Grid_B[0:grid_count], VNA_Grid_B[0:vna_grid_count], \
                        VEF_Grid_B[0:vef_grid_count])
{
#pragma acc parallel loop reduction(+:local_Ena,local_Eef,local_EH1,local_EXC0,local_EXC1)
  for (BN=0; BN<grid_count; BN++){
```

7.9 カ — Force.c

- 力 #2(運動エネルギー項 $\text{Tr}[\text{DM} \cdot \text{dH0}]$)、#4(VNA 格子項)、#5(重なり項 $-\text{Tr}[\text{EDM} \cdot \text{dS}]$)の縮約を OpenACC 化。共通ヘルパは「原子ごとのオフセット表+フラット化した (weight, hx, hy, hz)」を copyin し、`gang` (原子) × `vector reduction` (項) の 2 段縮約で力を得る。
- 力 #4B(射影 VNA)は意図的に CPU 維持。コメント原文: "In DC-LNO/GPUSOLVER runs many MPI ranks can share one GPU while GPUSOLVER still owns most device memory, so these transient copies can exhaust device memory. Keep Force4B on the CPU trace path"(Force.c:5588-5596)。
- diff 上の変更行数が最大(raw ±11,000 行超)だが、大半は全面再インデントで、実質変更は約 3,700 行。

Force.c:166-183(抜粋)— `gang`(原子) × `vector`(項)の 2 段縮約

```
#pragma acc parallel loop gang
for (Mc_AN = 0; Mc_AN <= num_atoms; Mc_AN++) {
  double sumx = 0.0; double sumy = 0.0; double sumz = 0.0;
  if (Mc_AN != 0) {
    size_t start = atom_offsets[Mc_AN - 1];
    size_t end = atom_offsets[Mc_AN];
    size_t idx;
#pragma acc loop vector reduction(+:sumx, sumy, sumz)
    for (idx = start; idx < end; idx++) {
      double weight = weights[idx];
      sumx += weight * vx[idx];
      sumy += weight * vy[idx];
      sumz += weight * vz[idx];
    }
  }
  Fx[Mc_AN] = sumx; Fy[Mc_AN] = sumy; Fz[Mc_AN] = sumz;
}
```

7.10 派生ソルバ群 — 分極・DMmu・CWF・LNO(定型パターン 11 ファイル)

以下の 11 ファイルには同一の定型パターンが適用されている: `openmx_gpuserver_dense_utils.h` の include、GPUSOLVER 要求時のデバイス割当、static コンテキスト、そして **ELPA2 分岐の直前**への `scf_eigen_lib_flag==GPUSOLVER && GPU_CPU_SWITCH_NUM<=n && na_rows==n && na_cols==n` 分岐の挿入(`OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` による対角化)。条件を満たさなければ従来の ELPA/ScaLAPACK に流れる。

ファイル	変更行数	GPU 対角化の挿入箇所
<code>Band_DFT_Col_CWF.c</code>	+74/-6	Zheevx × 3(S・H・CWF バンド)
<code>Band_DFT_Col_DMmu.c</code>	+75/-5	Zheevx × 4
<code>Band_DFT_NonCol_CWF.c</code>	+76/-11	Zheevx × 3
<code>Band_DFT_NonCol_DMmu.c</code>	+76/-5	Zheevx × 4
<code>Cluster_DFT_Col_CWF.c</code>	+64/-5	Dsyevx × 2

ファイル	変更行数	GPU 対角化の挿入箇所
Cluster_DFT_Col_DMmu.c	+63/-4	Dsyevx × 2
Cluster_DFT_Col_LNO.c	+66/-5	Dsyevx × 2
Cluster_DFT_NonCol_CWF.c	+65/-2	Dsyevx × 1 + Zheevx × 1
Cluster_DFT_NonCol_DMmu.c	+66/-3	Dsyevx × 1 + Zheevx × 1
Electric_Polarization_BandCol.c	+27/-5	Dsyevx × 1 + Zheevx × 4(DM・重なり・H・ABC)
Electric_Polarization_BandNonCol.c	+40/-14	Zheevx × 5(ELPA1 側フォールバック条件の調整も含む)

表 7-1: 定型 GPUSOLVER 分岐が挿入されたファイル群

7.11 汎用固有値ルーチン — Eigen_lapack.c / EigenBand_lapack.c

- `Eigen_lapack()` (実対称) は `flag != ELPA1/2 && n ≥ 1000` で `Eigen_gpusolver_x` (→ `gpusolver_Syevdx`) へ分岐。
Eigen_lapack を呼ぶ 20 以上のファイル(軌道最適化・DC・LNO など)が自動的に GPU 化の恩恵を受ける。上位に固有ベクトル直交性検査つきのソルバ巡回リトライがある。
- `EigenBand_lapack()` (エルミート) は $N \geq 1000$ で `gpusolver_zheevx` へ。失敗時は CPU の Householder 法 `Eigen_HH` にフォールバック。
- 両者とも OpenACC デバイス常駐配列を直接処理する `_openacc` エントリを持ち、非共線 DC の OpenACC 経路などから使われる。2 次元配列 ⇄ 1 次元デバイス配列の詰め替えも device 上で行う。

8. GPU 化以外の変更点

8.1 OpenMP バグ修正 — private 指定漏れによるヒープ破壊の解消

オリジナル OpenMX 4.0 の `Stress.c` と `Total_Energy.c` には、`#pragma omp parallel` の `private()` 節にループ変数 `i` が入っておらず、暗黙 `shared` の `i` を複数スレッドが同時に読み書きする競合があった。あるスレッドがループ境界を判定した直後に別スレッドが `i` を進めると配列の範囲外アクセスが起こり、ヒープ破壊(malloc メタデータの破損)として顕在化する。本 GPU 版の開発中に `runtest` のクラッシュ原因として特定され、修正された(コミット `d451eeb8`)。

Stress.c:729 — 修正内容(private リスト末尾への `i` 追加。**Total_Energy.c:2085** も同形)

```
/* 修正前(オリジナル)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5)

/* 修正後(GPU 版)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5,i)
```

- **Stress.c**: 競合箇所は並列リージョン内の `for (i=0; i<9; i++){ sumt1[i]=0.0; ... }` (原子ループ内で毎回実行)。このファイルの差分はこの 1 行のみ。
- **Total_Energy.c**: `Calc_EXC_EH1()` の Lebedev 球面グリッド積分並列区間。競合ループは `for (i=0; i<(FNAN[Gc_AN]+1); i++)` で `Dr[i][ir][ia]` 等を添字アクセスするため、競合時に確保域外を指し得る(ヒープ破壊経路)。

意義: この 2 件はオリジナル OpenMX 4.0 にも存在するバグであり、GPU と無関係に OpenMP スレッド並列(スレッド数 ≥ 2) で発症し得る。CPU 版 OpenMX を使う場合にも適用価値がある修正である。

8.2 メモリ管理・堅牢化

ファイル	修正内容
<code>openmx.c</code>	起動直後に <code>Raise_Stack_Limit()</code> を呼び、スタックのソフトリミットをハードリミットまで自動で引き上げる(<code>getrlimit/setrlimit(RLIMIT_STACK)</code>)。 <code>ulimit -s</code> 未設定環境での大きなスタック配列によるクラッシュ対策
<code>truncation.c</code> / <code>Free_Arrays.c</code>	<code>VNA_Grid</code> / <code>VNA_Grid_B</code> / <code>VEF_Grid</code> / <code>VEF_Grid_B</code> の free 後に NULL 代入を追加(二重 free 防止)
<code>Set_Nonlocal.c</code>	固定サイズ配列をオーバーフロー検査付きヒープ確保に置換 (§ 7.7)
<code>Free_Arrays.c</code>	終了時(wherefrom==0)に <code>openmx_gemmul8ReleaseWorkspaces()</code> を呼ぶ(GPU 関連)

openmx.c:75-88 — `Raise_Stack_Limit`(全文)

```
static void Raise_Stack_Limit(void)
{
#ifdef RLIMIT_STACK
    struct rlimit limit;

    if (getrlimit(RLIMIT_STACK,&limit)!=0) return;
    if (limit.rlim_cur==RLIM_INFINITY) return;
    if (limit.rlim_max==0 || limit.rlim_cur==limit.rlim_max) return;

    limit.rlim_cur = limit.rlim_max;
    setrlimit(RLIMIT_STACK,&limit);
#endif
}
```

8.3 その他の修正

ファイル	修正内容
<code>Runtest.c</code>	(1) 入力ファイル判定を <code>strstr</code> による部分一致から末尾一致 <code>has_dat_suffix()</code> に厳密化(<code>foo.dat~</code> 等の誤検出防止)。(2) 各テスト実行前に旧 <code><System.Name>.out</code> を削除し、前回結果の混入を防止(MPI_Barrier 付き)
<code>Spherical_Bessel.c</code>	ホットパスの呼び出し毎 <code>malloc/free</code> (漸化式係数配列)を除去し、係数をインライン計算化(数式は同一)
<code>openmx_common.h</code>	インクルードガードを追加(オリジナルには無かった)。バージョン文字列を "4.0" → "4.0 GPU" に変更
<code>Input_std.c</code>	GPU 関連キーワードの追加(第 10 章)
<code>mpao.c</code>	<code>#include <ctype.h></code> の 1 行差のみ(ビルド対象外のユーティリティで影響なし)

9. ビルドシステム

9.1 ツールチェーンの全面刷新

オリジナルの `makefile` (Intel oneAPI + MKL)は `Makefile` (NVHPC + CUDA + 自前 FFTW + GEMMu8)に 全面書き換えされた。原本は `makefile.cpu_intel.bak` としてバイト単位で同一保存されている。

項目	オリジナル(makefile)	GPU 版(Makefile)
C コンパイラ	<code>mpiicx -O3 -xHOST -qopenmp</code>	NVHPC 同梱 OpenMPI の <code>mpicc (nvc) + -acc -Minfo=accel -std=c11 -mtune=native -march=native -fopenmp</code>
Fortran	<code>mpiifx</code>	<code>mpif90 (nvfortran) -O2 -fopenmp</code>
BLAS/LAPACK	MKL(静的)	NVHPC 同梱の静的 <code>liblapack_lp64.a</code> / <code>libblas_lp64.a</code>
ScaLAPACK	MKL ScaLAPACK	HPC-X OpenMPI 側の <code>libscalapack_lp64.a</code>
FFT	MKL FFTW ラッパ	自前ビルドの静的 FFTW 3.3.11(third_party)
GPU ライブラリ	—	<code>-lcudart -lcusolver -lcublas -lcublasLt -lcuda -lnvidia-ml -lnvjitlink -cuda + libgemmu8.a</code>
SDK 前提	Intel oneAPI	<code>NVHPC_ROOT ?= /opt/nvidia/hpc_sdk/Linux_x86_64/26.3</code> 、 <code>NVHPC_CUDA_VERSION ?= 13.1</code> (変数で上書き可)

ビルド環境の隔離: `Makefile:69-104` で `PATH` を NVHPC のみに固定し、`CPATH` / `LD_LIBRARY_PATH` / `CC` など 30 個超の環境変数を `unexport` する。ユーザー環境の `CFLAGS` 等は効かない(意図的)。

Makefile:131-140 — コンパイラ定義(抜粋)

```
COMMON_INCLUDES = $(FTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include -I$(NVHPC_CUDA_TARGET)/include -I$(NVHPC_CUDA_HOME)/include
COMMON_CFLAGS   = -w -Minfo=accel -acc -std=c11 -mtune=native -march=native -fopenmp $(COMMON_INCLUDES)

CC = $(MPICC) $(COMMON_CFLAGS) -O3
CC2 = $(MPICC) $(COMMON_CFLAGS) -O1
CC3 = $(MPICC) $(COMMON_CFLAGS) -O2
CC_NOACC = $(filter-out -Minfo=accel -acc,$(CC2))
CC_NOACC_O3 = $(filter-out -Minfo=accel -acc,$(CC))
CC_DFTD3 = $(MPICC) -w -O0 -std=c11 -fopenmp $(COMMON_INCLUDES)
FC = $(MPIF90) -O2 -mtune=native -march=native -fopenmp $(FTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include
```

オリジナルは全ファイル `$(CC)` 一本だったのに対し、GPU 版は**ファイル別の最適化・OpenACC 調整**を行う (表 6-1)。OpenACC のターゲットアーキ指定(`-gpu=ccXX`)はなく、ビルドマシンの GPU を自動検出する。クロスターゲットしたい場合は `COMMON_CFLAGS` に追加する。

9.2 リンク構成

Makefile:142-150, 172(抜粋)

```
NVPL_SCALAPACK_LIB ?= $(NVHPC_MPI_LIBDIR)/libscalapack_lp64.a
NVPL_LAPACK_LIB    ?= $(NVHPC_COMPILER_LIB)/liblapack_lp64.a
NVPL_BLAS_LIB      ?= $(NVHPC_COMPILER_LIB)/libblas_lp64.a
NVPL_LIBS          ?= -Wl,--start-group $(NVPL_SCALAPACK_LIB) $(NVPL_LAPACK_LIB) $(NVPL_BLAS_LIB) -Wl,--end-group
CUDA_LIBS          ?= ... -Wl,--no-as-needed -lnvjitlink -Wl,--as-needed -lcudart -lcusolver -lcublas -cuda
LIB = $(FTW3_LIBS) $(NVPL_LIBS) $(MPI_FORTTRAN_LIBS) -pgf90libs -mp -lpthread -lm -ldl $(CUDA_LIBS)
...
LIB += -lcublasLt -lcuda -lnvidia-ml -lstdc++
```

リンク行は `$(CC) $(OBJ) $(GEMMUL8_LIB) $(LIB) -lm -o openmx (Makefile:337-338)`。OBJ への追加分：
`Total_Energy_GPU.o`、`gemmul8_bridge.o`、`gpusolver_Syevd.o` `gpusolver_Syevdx.o` `set_cuda_default_device_from_local_rank.o`
`set_openacc_device_from_local_rank.o`、`OutData_Binary.o` (呼び出し元の無いデッドリンク)。かつて含まれていた
`my_cublasDgemm.o` `my_cublasZgemm.o` は `a7f8cba6` で削除された。`openmx_cuda_kernels.o` は含まれない (§ 6.4)。

9.3 third_party の自動ビルド

既定ターゲットが `openmx: $(FFTW3_LIB) $(GEMMUL8_LIB) $(OBJ)` のため、素の `make` で FFTW3 → GEMMUL8 → 本体の順に
 全自動でビルドされる。両者とも git submodule である (`.gitmodules`: RIKEN-RCCS/GEMMUL8, FFTW/fftw3)。

FFTW 3.3.11

- submodule の git ソースには生成済み codelets が無いため、番兵ファイルの有無を確認し、無ければ
`https://fftw.org/fftw-3.3.11.tar.gz` (リポジトリにもコミット済み)から `rsync --ignore-existing` で不足ファイルのみ補
 完する。
- タグ `fftw-3.3.11` ちょうどのチェックアウトかを検証し、必要なら `autoreconf`。out-of-tree(`build/fftw3`)で `configure -
 -enable-static --disable-shared` (コンパイラは `nvc/nvc++/nvfortran`)→ `make install` で `third_party/fftw3-install`
 へ。

GEMMUL8

- コミット `884a2875...` に固定 fetch/checkout(理由は § 4.7)。上流のビルドシステムは使わず、`gemmul8_openmx.cu` 1 ファ
 イルを `nvcc` でコンパイルして `ar rcs` で `libgemmul8.a` を自前生成する。

Makefile:716-727 — GEMMUL8 の取得と単一 TU ビルド

```
$(GEMMUL8_DIR)/Makefile:
    mkdir -p $(GEMMUL8_REPO_DIR)
    cd $(GEMMUL8_REPO_DIR) && \
    { [ -d .git ] || git init -q .; } && \
    { git remote get-url origin >/dev/null 2>&1 || git remote add origin $(GEMMUL8_REPO_URL); } && \
    git fetch --depth 1 origin $(GEMMUL8_COMMIT) && \
    git checkout -q $(GEMMUL8_COMMIT)
$(GEMMUL8_OPENMX_OBJ): gemmul8_openmx.cu $(GEMMUL8_DIR)/Makefile
    $(NVCC_GEMMUL8_ENV) $(NVCC) $(NVCC_GEMMUL8_FLAGS) -c gemmul8_openmx.cu -o $(GEMMUL8_OPENMX_OBJ)
$(GEMMUL8_LIB): $(GEMMUL8_OPENMX_OBJ)
    mkdir -p $(GEMMUL8_DIR)/lib
    ar rcs $(GEMMUL8_LIB) $(GEMMUL8_OPENMX_OBJ)
```

`clean` は `fftw3-clean` (`build/fftw3` と `fftw3-install` を削除)と `gemmul8-clean` (`libgemmul8.a` を削除)を含むよう拡張され
 た。つまり `make clean` 後は `third_party` の再ビルドが必要になる。

9.4 ELPA(elpa-2018.05.001)

`elpa-2018.05.001/` ディレクトリはオリジナルと完全同一(diff 差分なし)。オブジェクトリストも同一で、カーネルは generic
 CPU 版のまま(ELPA の GPU カーネルは使っていない)。違いは Fortran コンパイラが `mpiifx` → `nvfortran` になった点の
 み。GPU 版でも `n < 1000` の系や GPU 条件を満たさない経路では ELPA が現役で使われるため、この維持は重要である。

9.5 派生 Makefile

- `makefile.cpu_intel.bak` — オリジナル makefile のバイト同一コピー(CPU/Intel 構成の保存)。

- **Makefile.bunshiken** — 共用計算機(モジュール環境: `nvhpc/25.9-nompi` + `openmpi/5.0.8` + `gcc-toolset/15`)向けの変種。NVHPC 25.9 + 外部 OpenMPI、CUDA 13.0、nvcc ホストコンパイラが g++、GEMMu8 が旧レイアウト(コミット固定なし)、オブジェクト名が gpusolver リネーム前の `cusolver_*.o` のまま、という点で主 **Makefile** より一世代古い。他環境へ移植する際の参考例。

9.6 ビルド手順と注意

1. `git clone --recursive` (submodule: fftw3, GEMMu8。GEMMu8 は無くても make が自動 fetch する)。
2. `source/` で `make` (既定ゴール openmx)。FFTW3 の補完→configure→install、GEMMu8 のコンパイル→ar、本体のコンパイル→リンクまで一括実行される。
3. `make install` で `../work/openmx` にコピー。ユーティリティ群は `make all`。

ビルド時の注意: (1) NVHPC のバージョンを変えると § 6.5 のミスコンパイル回避策の再検証が必要。(2) HPC-X のパスにはバージョン文字列(hpcx-2.25.1)がハードコードされており、SDK 更新時は修正が要る。(3) GEMMu8 のアーキは `make GEMMu8_GPU_ARCH=90` のように上書き可能(既定は `nvidia-smi` 自動検出)。(4) オリジナルにあった API ライブラリ (`libopenmx_api`)・`elpa_col` などの補助ターゲットは削除されている。(5) 並列 `make(-j)` は依存関係上は可能な構成だが、本プロジェクトの運用では明示指定時のみ使用している。

10. 実行時制御リファレンス

10.1 入力ファイルキーワード

GPU 機能に関して追加・拡張された入力キーワードは次の 2 つだけである(それ以外の調整はすべて環境変数)。

Input_std.c:777-791 — 追加部分(全文)

```
/* "cusolver" is a deprecated alias of "gpusolver"; ... */
s_vec[0]="elpa2"; s_vec[1]="lapack"; s_vec[2]="elpa1"; s_vec[3]="gpusolver"; s_vec[4]="cusolver";
i_vec[0]=ELPA2; i_vec[1]=0; i_vec[2]=ELPA1; i_vec[3]=GPUSOLVER; i_vec[4]=-GPUSOLVER;
input_string2int("scf.eigen.lib", &scf_eigen_lib_flag, 5, s_vec,i_vec);
if (scf_eigen_lib_flag==GPUSOLVER){
    if (myid==Host_ID){
        printf("Warning: scf.eigen.lib=cusolver is deprecated; use scf.eigen.lib=gpusolver instead.\n");
    }
    scf_eigen_lib_flag = GPUSOLVER;
}
scf_eigen_lib_flag_input = scf_eigen_lib_flag;

/* use GPU? (added by H.Kawai, February 2024) */
input_int("scf.Gpu.Num", &SCF_Gpu_Num, 30);
```

キーワード	型	既定値	意味
scf.eigen.lib	文字列選択	elpa2	従来の elpa2 / lapack / elpa1 に gpusolver (GPU 経路一式の有効化)を追加。 cusolver は警告付きの非推奨エイリアス。GPU 不在時は自動的に ELPA2 に降格
scf.Gpu.Num	int	30	ノード内で使用する GPU 台数の上限。現在は DC-LNO の GPU 台数切り詰めのみ使用(既定 30 は実質「検出された全 GPU を使う」)

10.2 環境変数一覧

GPU 実行のチューニングは環境変数で行う設計である(入力ファイルを変えずにジョブスクリプト側で調整できる)。**OPENMX_**接頭辞版と **GEMMUL8_** 接頭辞版がある変数は **OPENMX_** 版が優先される。

環境変数	既定値	意味(定義箇所)
GEMMUL8(gemmul8_bridge.cu)		
(OPENMX_)GEMMUL8_DISABLE / _D / _Z	off	GEMMUL8 を無効化し cuBLAS FP64 に固定(全体 / 実数のみ / 複素のみ)
(OPENMX_)GEMMUL8_NUM_MOD_D / _Z	15	法の個数(精度)。範囲 2~20、範囲外は 15 に戻る
(OPENMX_)GEMMUL8_FASTMODE_D / _Z	0	高速スケーリングモード(精度低下)
(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT	30	ワークスペースの総 VRAM 比上限(Band_DFT_NonCol は未設定時 50 を setenv)
(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB	1536	ワークスペース確保後に残すべき最低空き VRAM
バンド計算の並列度・常駐制御(Band_DFT_Col.c / Band_DFT_NonCol.c)		
OPENMX_GPUSOLVER_SERIAL_GPU_TURNS	1	GPU ターン直列化の有効/無効(0 で並行投入)
OPENMX_BAND_GPU_TURN_GROUP	4	同時に走ってよい GPU ターン数(グループ幅)
OPENMX_BAND_GPU_MAX_CONCURRENT_K	4	all_knum#1 の第 1 対角化ループの同時 k 数。8c7801fa で上限 4 にクランプ。 NonCol では指定値にも適応制限(§ 5.5.2)が掛かる
OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE	—	上記 2 ノブの共通フォールバック値(1 GPU あたり同時ランク数)
OPENMX_BAND_GPU_PERSISTENT	0	SCF 反復間のデバイスバッファ常駐(共有 GPU では非推奨)

環境変数	既定値	意味(定義箇所)
<code>OPENMX_BAND_GPU_PERSISTENT_MIN_FREE_MB</code>	2048	常駐を許す最低空き VRAM
<code>OPENMX_BAND_GEMMUL8_TURN_RELEASE</code>	1	ターン毎の GEMMUL8 ワークスペース解放(0 で保持)
<code>OPENMX_BAND_PROFILE</code>	0	性能計測カウンタの表示
DC 法(Divide_Conquer.c)		
<code>OPENMX_DC_GPU_THRESHOLD</code>	1000	GPU 化する局所クラスタ次元の下限
<code>(OPENMX_)GPUSOLVER_MIN_FREE_AFTER_MB</code>	1536	固有値ソルバ用確保後の最低空き VRAM(preflight)
<code>OPENMX_CUBLAS_MIN_FREE_AFTER_MB</code>	—	cuBLAS 用の同上
行列要素構築のオプトイン(Set_Hamiltonian.c / Set_OLP_Kin.c)		
<code>OPENMX_SETHAM_BASE_GPU</code>	0	H の base 結合を GPU 実行(実測では CPU が速い)
<code>OPENMX_OLPKIN_RADIAL_GPU</code>	0	重なり・運動行列要素の動径積分を GPU 実行(スレッド数 1 必須。転送律速で通常は CPU が速い)

表 10-1: GPU 関連環境変数の一覧

10.3 チューニング指針

1. **まず既定値で走らせる** — 既定値は「16 GB 級 GPU を複数ランクで共有」する構成に合わせて保守的に調整されている。
2. **stderr を確認する** — "GEMMUL8 workspace fallback" 警告が出ていれば VRAM 不足で FP64 に落ちている。ランク数を減らす、`OPENMX_BAND_GPU_TURN_GROUP` を小さくする、`NUM_MOD` を下げる、などで対処。
3. **GPU を占有できる場合** — `OPENMX_BAND_GPU_PERSISTENT=1`、`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS=0` (または `TURN_GROUP` を大きく) で転送・同期を減らせる余地がある。
4. **再現性を最優先する場合** — `OPENMX_GEMMUL8_DISABLE=1` で GEMM を FP64 に固定すると、メモリ状況によるエンジン切替の影響を排除できる (§ 4.8)。
5. **DC/DC-LNO** — 局所クラスタが小さい系では GPU 化対象が少ない。`OPENMX_DC_GPU_THRESHOLD` を下げると対象は増えるが、小行列では転送コストが勝ちやすい。

11. 制約・注意点の総まとめ

11.1 ビルド・実行環境の前提

- このツリーは CPU 専用にビルドできない(CUDA ヘッダを無条件 include)。CPU 版が必要ななら オリジナルまたは `makefile.cpu_intel.bak` を使う。
- NVHPC 26.3 + CUDA 13.1 が既定。バージョン変更時は § 6.5 のミスコンパイル回避一覧を再検証すること。
- GEMMu8 は INT8 Tensor Core 必須・単一アーキビルド。GPU 世代を変えたら再ビルド。
- GPU 実行時は MPI のみ(OpenMP スレッド 1)の運用が想定されている。`Set_OLP_Kin` の GPU 経路は `Nthrds==1` を明示的に要求する。

11.2 メモリ設計の目安

- 1 ランクあたりの VRAM 需要(バンド計算・次元 n): 複素密行列 $16n^2$ バイト × 数本 + syevdx ワークスペース + GEMMu8 ワークスペース(約 1 GiB、複素は実数の約 3 倍)。
- 共有 GPU では「同時ランク数 × 上記」が空き VRAM に収まる必要がある。Band 系は自動制御(ターン直列化・適応制限)が働くが、Cluster root-dense や DC-LNO direct dispatch には同等の自動絞り込みはない(DC は preflight あり)。

11.3 数値再現性

- ビット単位の再現を壊し得る要因: (1) GEMMu8 ⇄ cuBLAS FP64 のメモリ状況による切替、(2) scatter-add の `acc atomic /atomicAdd`(加算順非決定)、(3) S の小固有値処理の経路差、(4) GEMMu8 の環境変数設定差(`num_moduli / fastmode`)。
- 再現に配慮した設計: DM 構築の `loop seq` 逐次縮約、GEMMu8 自体の bitwise 再現性、ベッセルテーブルの bit 一致 キャッシュ、(未結線だが)openmx_cuda_kernels.cu の逐次縮約カーネル。
- 検証時は `OPENMX_GEMMU8_DISABLE=1` と十分な VRAM 余裕で条件を固定するのが確実。

11.4 ハング・異常終了

- `wait_cudafunc` は恒久エラーで無限ループする(§ 3.6)。「GPU 実行が止まって見える」ときの第一容疑者。
- 固有値計算失敗時の挙動は経路により abort / CPU フォールバック / 素通りと異なる(表 5-2)。
- collective 版デバイス割当関数は全ランカー斉呼び出しが前提(片側だけ呼ぶとデッドロック)。

11.5 AMD GPU など他ベンダーへの移植観点

- gpusolver 命名は準備済みだが、切替はマクロ等で機構化されていない。cuSOLVER 依存点は `gpusolver_Syevd(x).c` と各ソルバの常駐コンテキスト(Xsyevdx 直接呼び出し)に集約されている。
- cuBLAS 依存点は Ddgm / Zdgm と GEMMu8 ブリッジ(内部 INT8 GEMM とフォールバック GEMM)。GEMMu8 は上流に HIP/ROCm バックエンドがあるが、本統合(単一 TU 実体化)は CUDA 経路のみを実体化している。
- OpenACC は NVHPC 依存(AMD なら OpenMP target offload などへの書き換えが必要)。`wait_cudafunc` の「戻り値 0 = 成功」前提も移植時の確認点。

11.6 オリジナルから挙動が変わる非 GPU 項目

- OpenMP 並列時のヒープ破壊バグ 2 件が修正済み(§ 8.1) — オリジナルにも存在するバグ。

- 起動時にスタックのソフトリミットが自動で引き上げられる(§ 8.2)。
- runtest の判定・実行手順が厳密化されている(§ 8.3)。
- バージョン文字列は "4.0 GPU"。

11.7 休眠・準備コード一覧(改修時の誤解防止)

コード	状態と位置づけ
<code>gpusolver_Syevd</code> (Xsyevd 版)	呼び出し 0 件(Syevdx 版に統一された)。旧式の「大域 rank % デバイス数」割当が残る
<code>LU_Zinverse_GPU</code>	実装完了・呼び出しコメントアウト。NEGF 表面グリーン関数の GPU 化布石(§ 5.7)
<code>openmx_cuda_kernels.cu/.h</code>	bitwise 再現検証済みの生 CUDA カーネル集。git 未追跡・ビルド対象外・呼び出し 0 件(§ 6.4)
<code>ClusterCol_GpuSolverDensePath</code>	到達不能の旧経路(root-dense 経路が先に goto する)
DMmu/CWF/LNO 系の GEMMu8 ヘルパ、 <code>BandCo</code> <code>l_GpuSolver_Zgemm_OpenACC</code> 、 <code>ClusterCol_GEMMu8Dge</code> <code>mm_OpenACC</code>	定義のみ(将来の GEMM GPU 化の準備)。なお <code>my_cublasDgemm/Zgemm</code> は <code>a7f8cba6</code> で削除済み
DC-LNO の GPU プロキシ機構	<code>DCLNO_ENABLE_SHARED_GPU_PROXY 0</code> で温存(実験用)
<code>OutData_Binary.o</code> / <code>getDeviceCount()</code> 宣言	デッドリンク / デッド宣言

表 11-1: 休眠・準備コードの一覧

付録 A. 変更ファイル一覧(37 ファイル)

+n/-m は生の diff 行数。全面再整形されたファイルは空白・空行無視の実質変更行数を括弧内に付記。分類: **GPU**=GPU 化、**OMP**=OpenMP 修正、**FIX**=バグ修正・堅牢化、**OPT**=CPU 最適化、**BLD**=ビルド、**MISC**=その他。

ファイル	+n/-m(実質)	分類	変更概要
Band_DFT_Col.c	+4,116/-2,491(3,667)	GPU	コリニアバンドの全面 GPU 化: 密行列構築キャッシュ、Xsyevdx 対角化、GEMMu8 三重積、S 変換キャッシュ、GPU ターン直列化、all_knum 両経路
Band_DFT_NonCol.c	+3,260/-91(2,736)	GPU	非コリニアバンドの GPU 化: OpenACC 常駐 root-dense 経路、VRAM 見積りによる適応並列度制限
Cluster_DFT_Col.c	+1,016/-12(859)	GPU	root-dense GPU 対角化経路、S 変換のデバイスキャッシュ、packed キャッシュからの行列構築、固有ベクトル分配
Cluster_DFT_NonCol.c	+1,706/-218(1,252)	GPU	root-dense 対角化(Dsyevx/Zheevx)、GEMMu8 × 12、固有ベクトルキャッシュと scatter、DM/EDM 縮約
Divide_Conquer.c	+4,399/-2,978(2,223)	GPU	DC 法の GPU 化: 原子別クラスタの Xsyevdx、GEMMu8→cuBLAS→CPU 多段 GEMM、VRAM preflight と sticky 無効化
Divide_Conquer_LNO.c	+1,934/-2,007(1,879)	GPU	DC-LNO の GPU 化: direct per-rank dispatch(プロキシ機構は温存)、しきい値 800
Krylov.c	+366/-16	GPU	スレッド別 GPU ワークスペースプール、dgemm 4 箇所→GEMMu8、部分空間対角化→gpusolver
Band_DFT_Col_CWF.c / ほか定型 9 ファイル(§ 7.10)	各 +63~76/-2~11	GPU	ELPA2 分岐手前への定型 GPUSOLVER 分岐挿入(dense_utils 経由 Dsyevx/Zheevx)
Electric_Polarization_BandCol.c / _BandNonCol.c	+27/-5、+40/-14	GPU	分極計算の対角化 5 箇所ずつに GPUSOLVER 分岐
Eigen_lapack.c	+143/-8	GPU	n≥1000 で gpusolver_Syevdx へ分岐、OpenACC 直結版を追加。呼び出し元 20+ ファイルが自動恩恵
EigenBand_lapack.c	+112/-3	GPU	複素版: gpusolver_zheevx 分岐、失敗時 Eigen_HH フォールバック
Lapack_LU_inverse.c	+134/-4	GPU	cublasZgetrf/ZgetriBatched による LU_Zinverse_GPU を追加(呼び出しはコメントアウト=休眠)
Set_Hamiltonian.c	+1,999/-1,149(1,784)	GPU	格子積分の OpenACC 版(現在無効)+GPUSOLVER 用 packed H/S キャッシュ API 新設+メモリ Allreduce 判定
Set_Nonlocal.c	+1,356/-306	GPU/FIX	動径 k 積分の OpenACC 化+固定配列のヒープ化(オーバーフロー検査付き)
Set_OLP_Kin.c	+529/-181	GPU/OPT	動径積分の OpenACC 化(オプトイン)+bit 一致ベッセルテーブルキャッシュ
Total_Energy.c	+319/-115	GPU/OMP	EH0 二中心項バッチ・EXC/EH1・双極子・CWF-Dc の GPU 縮約呼び出し+OpenMP private 修正
Total_Energy_GPU.c (新規扱い)	517 行	GPU	Total_Energy 用 OpenACC カーネル集(-acc 分離ビルドのため別 TU)
Force.c	+11,522/-11,115(3,672)	GPU	力 #2/#4/#5 の OpenACC 縮約(#4B は CPU 維持)。raw 行数の大半は全面再インデント
Stress.c	+1/-1	OMP	OpenMP private リストに i を追加(ヒープ破壊修正)
DFT.c	+172/-4	GPU	GPU デバイス初期化、OpenACC 実行ランク選抜、packed キャッシュ準備、NonCol 固有ベクトル scatter
Input_std.c	+15/-3	GPU	scf.eigen.lib への gpusolver 追加(cusolver は非推奨エイリアス)、scf.Gpu.Num 追加

ファイル	+n/-m(実質)	分類	変更概要
<code>openmx_common.h</code>	+93/-13	GPU/MISC	CUDA ヘッダ取込、GPUSOLVER enum、GPU_CPU_SWITCH_NUM、wait_cudafunc/cudacall、GPU 系プロトタイプ、include ガード、Version "4.0 GPU"
<code>tran_prototypes.h</code>	+10/-0	GPU	CUDA_CHECK マクロ追加
<code>openmx.c</code>	+16/-0	FIX	Raise_Stack_Limit()(スタック上限自動引き上げ)
<code>Runtest.c</code>	+19/-7	FIX	.dat 判定の厳密化、テスト前の旧 .out 削除
<code>Free_Arrays.c</code>	+23/-17	GPU/FIX	終了時の GEMMu8 ワークスペース解放+free 後 NULL 代入
<code>truncation.c</code>	+14/-10	FIX	VNA/VEF グリッドの free 後 NULL 代入(二重 free 防止)
<code>Spherical_Bessel.c</code>	+12/-24	OPT	ホットパスの毎呼び出し malloc/free を除去
<code>mpao.c</code>	+0/-1	MISC	include 1 行差のみ(ビルド対象外)
<code>makefile</code> → <code>Makefile</code>	+360/-374	BLD	Intel oneAPI+MKL 構成から NVHPC+CUDA+自前 FFTW+GEMMu8 構成へ全面書き換え(第 9 章)

補足: `Set_ProExpn_VNA.c` は開発中に一度 GPU 化されたのち CPU 版へ戻されており(コミット 17→31)、現行ツリーではオリジナルと差分がない。`elpa-2018.05.001/` 以下も完全同一。削除されたソースファイルはない。

付録 B. 新規追加ファイル一覧

B.1 新規ソース(13 ファイル、計 2,555 行)

ファイル	行数	役割
<code>gpusolver_Syevdx.c</code>	387	cusolverDnXsyevdx による部分固有値分解ラッパ(実/複素 × ホスト/OpenACC の 4 変種)。GPU 対角化の実体
<code>gpusolver_Syevd.c</code>	131	cusolverDnXsyevd 版ラッパ(NVIDIA CUDA Library Samples 由来)。現在は未使用
<code>openmx_gpusolver_dense_utils.h</code>	79	密対角化の共通ディスパッチヘルパ(0/1-based 変換+失敗時 MPI_Abort)。11 ファイルが使用
<code>gemmul8_bridge.cu</code>	393	GEMMul8 呼び出しブリッジ: ワークスペースキャッシュ、環境変数制御、cuBLAS フォールバック
<code>gemmul8_openmx.cu</code>	67	GEMMul8 テンプレートの単一 TU 明示的実体化(4 シンボルのみ)
<code>Total_Energy_GPU.c</code>	517	全エネルギー計算の OpenACC カーネル集(EXC/EH1・双極子・CWF-Dc・EH0 パッチ)
<code>set_cuda_default_device_from_local_rank.c/.h</code>	73+9	ノード内 local rank による CUDA デバイスのラウンドロビン割当
<code>set_openacc_device_from_local_rank.c/.h</code>	79+12	同上の OpenACC 版(acc_set_device_num)
<code>LU_Zinverse_GPU.h</code>	9	GPU 版複素 LU 逆行列の宣言(休眠)
<code>openmx_cuda_kernels.cu/.h</code>	696+103	OpenACC 領域を生 CUDA 化した検証済みカーネル集(未結線・ビルド対象外。§ 6.4)

補足: 過去に存在した汎用 GEMM ラッパ `my_cublasDgemm.c` / `my_cublasZgemm.c` (49+63 行、呼び出し元なし)はコミット `a7f8cba6` で削除された (§ 3.5)。

B.2 ビルド・文書ファイル

ファイル	役割
<code>Makefile</code> (1,373 行)	新ビルドシステム本体。各種スパコン向け構成例のコメント付き。NVHPC 26.3 + CUDA 13.1 既定
<code>Makefile.bunshiken</code> (1,366 行)	共用計算機向け変種(NVHPC 25.9-nompi + OpenMPI 5.0.8 + gcc-toolset-15)。一世代古い構成
<code>makefile.cpu_intel.bak</code> (1,383 行)	オリジナル makefile のバイト同一バックアップ(Intel CPU 構成)
<code>third_party/</code>	GEMMul8(submodule、コミット 884a287 固定)、fftw3(submodule、タグ fftw-3.3.11)、fftw-3.3.11.tar.gz(codelets 補完用)

付録 C. 開発履歴(全 51 コミット)

#	ハッシュ	題名(原文)
1	0cf2acbc	Initial commit
2	60b6aa1e	Performed GPU acceleration
3	2f94ca06	Performed GPU acceleration on Divide_Conquer_LNO.c
4	e3fc656e	Optimized Band_DFT_NonCol.c
5	24e8b34f	Optimized Cluster_DFT_Col.c
6	33e9459b	Optimized Cluster_DFT_Col.c
7	82579d12	Bug fix
8	33a07450	Fixed bugs in Band_DFT_Col.c and Cluster_DFT_Col.c
9	b4221bba	Fixed a bug in runtestL
10	18d92700	Bug fix
11	563f69bd	Add files that were not included in the commit
12	1b020264	Optimized Band_DFT_Col.c and Band_DFT_NonCol.c
13	f4041307	Optimized Set_Hamiltonian.c
14	9f1633da	Optimized Set_OLP_Kin.c
15	d6b463a8	Explicitly indicate that the generation of overlap matrices is GPU-accelerated
16	5383544b	Optimized Total_Energy.c
17	c9e111ea	Optimized Set_ProExpn_VNA.c
18	8162b8ec	Optimized Set_ProExpn_VNA.c and Total_Energy.c
19	91416fb1	Bug fix
20	24a113a9	Add files that were not included in the commit
21	87ea7062	Bug fix
22	faf76fa2	Bug fix
23	a8224724	Add .gitignore for build artifacts and runtime output
24	d3c45cb3	Optimized Bnad_DFT_NonCol.c
25	db7b9327	Add files that were not included in the commit
26	a61f24b3	Bug fix
27	d6d19f99	Add README.md
28	a91f920f	Bug fix
29	88b135f4	Optimized Krylov.c
30	34cf8bc1	Optimized Divide_Conquer_LNO.c
31	685a2bc9	Reverted Set_ProExpn_VNA.c to the CPU version
32	2aa31a43	Add files that were not included in the commit
33	d3effce1	Bug fix of Divide_Conquer.c
34	34b52269	GPU acceleration for paths where all_knum !=1 in Band_DFT_Col.c
35	75ef67c7	GPU acceleration for paths where all_knum !=1 in Band_DFT_NonCol.c
36	1e0c777e	Remove runtest.result from Git

#	ハッシュ	題名(原文)
37	c8c1e3e1	Modify the Makefile
38	cb4d348d	Now compatible with the latest GEMMu8
39	d451eeb8	Fixed OpenMP bugs in Total_Energy.c and Stress.c
40	2302f965	The name has been changed from CuSOLVER to GPUSOLVER
41	d5b897fd	Band_DFT_Col: serialize GPU turns in pairs and release GEMMu8 workspace per turn
42	788acb4c	Delete unnecessary files, such as the MAGMA library
43	d8219384	Rename the identifier from cusolver to gpusolver
44	8647a5ed	Set large2_example and Large3_example to be processed on the GPU.
45	f508a2a6	Enable direct GPUSOLVER dispatch for DC-LNO
46	3545a9d5	Edit README.md
47	ca3d97c2	Edit README.md
48	43e30c65	Limit band GPU solver concurrency adaptively
49	66ce1e62	Use GEMMu8 for remaining GPU GEMM paths
50	a7f8cba6	Remove obsolete cuBLAS wrapper files
51	8c7801fa	Limit band GPU k-turn concurrency

参考文献

1. OpenMX 公式サイト: <http://www.openmx-square.org/> (コード本体・マニュアル・入力キーワード)
2. GEMMu18(GEMMulate)リポジトリ: <https://github.com/RIKEN-RCCS/GEMMu18> (MIT ライセンス、責任開発者 Yuki Uchino, RIKEN R-CCS)
3. Y. Uchino, K. Ozaki, T. Imamura, "High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines", SC Workshops '25, doi:10.1145/3731599.3767539
4. K. Ozaki, Y. Uchino, T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", arXiv:2504.08009
5. Y. Uchino, S. Ma, T. Imamura, K. Ozaki, T. Gutsche, "Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem", arXiv:2512.08321(複素対応の理論)
6. Y. Uchino, K. Ozaki, T. Imamura, "Double-Precision Matrix Multiplication Emulation via Ozaki-II Scheme with FP8 Quantization", arXiv:2603.10634
7. H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", Int. J. High Perform. Comput. Appl. (2024), doi:10.1177/10943420241239588 (ozIMMU / Ozaki Scheme I 系の先行研究)
8. NVIDIA cuSOLVER Documentation(cusolverDnXsyevd/Xsyevdx): <https://docs.nvidia.com/cuda/cusolver/>
9. NVIDIA cuBLAS Documentation: <https://docs.nvidia.com/cuda/cublas/>
10. NVIDIA HPC SDK Documentation(OpenACC): <https://docs.nvidia.com/hpc-sdk/>
11. ELPA(Eigenvalue solvers for Petaflop Applications): <https://elpa.mpcdf.mpg.de/>
12. FFTW: <https://www.fftw.org/>

本書について: 本書は 2026-07-05 時点のソースツリー(コミット `8c7801fa`)と、オリジナル OpenMX 4.0 との全ファイル diff、および開発リポジトリの全コミット履歴の調査に基づいて作成された。記載の行番号・既定値は当該時点のものである。誤りを見つけた場合はソースコードを一次情報として優先されたい。